



Documentation

Creators

- Lucas Gomes Cecchini
- Gabriel Henrique Pereira

Overview

This document will help you use the Assets **Unity MMD Collection**.

With it you have tools that can facilitate the use of **MMD models** in **Unity**, as well as custom **Shaders** for this.

The **Shaders** were made based on studies in the [MMD4Mecanim package](#), being recreated and improved in two tools: The [Shader Graph](#) and [Amplify Shader Editor](#) due to the capabilities and applications of each resource.

It also has **Scripts** that shape the Unity interface to make it similar to the **MMD tools** and make it more practical and faster to use. In addition to other things that allow you to speed up development.

Instructions

You can get more information in the Playlist on YouTube:

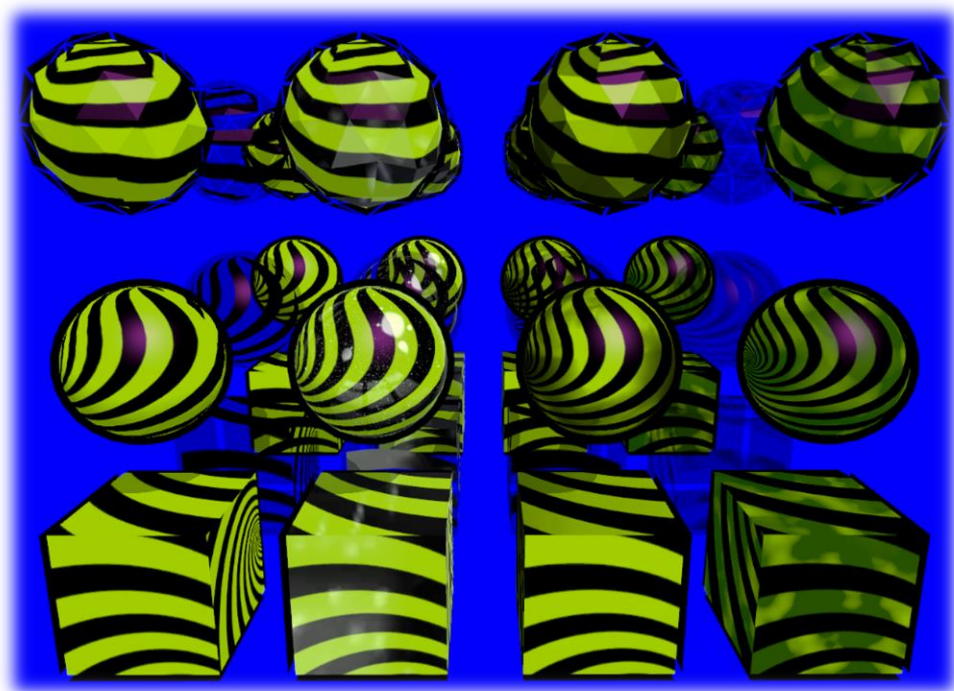
<https://youtube.com/playlist?list=PL5hnfx09yM4IWSWveW0NKCFX1Anec4dw7&si=UIqEtAoFcXZH7WFM>

Compatibility

This package is compatible with:

- Unity 6000.4.2f1
- Amplify Shader Editor v 1.9.9.9
- MMD4Mecanim 2020-01-05

Explanation of the Shader



MMD Shader

Add Sphere (Material)
MMD Collection/URP/MMD (Amplify Shade)
Edit...

MMD Data
Show Default Systems ☒
Mat-Name (JP) 球を追加 (EN) Add Sphere

Surface Options
Surface Type Opaque
Render Face Front
Depth Write Force Enabled
Depth Test LEqual
Alpha Clipping
Cast Shadows
Receive Shadows

Default GUI
Material Color
Diffuse
Specular
Ambient
Opaque
Reflection
Rendering
S-Shad
Edge (Outline)
On
Color
Size
Texture (Memo)
Effects
Texture
Toon
SPH
UV Layer
SPH SubTex
Tiling
Offset
Custom Effects Settings
Specular Intensity
SPH Opacity
Shadow Luminescence
HDR
Toon Tone
Multiple Lights
Fog
Render Queue
Enable GPU Instancing
Double Sided Global Illumination
Global Illumination

Add Sphere (Material)
MMD Collection/URP/MMD (Amplify Shade)
Edit...

MMD Data
Show Default Systems
Mat-Name (JP) 球を追加 (EN) Add Sphere

Material Color
Diffuse
Specular
Ambient
Opaque
Reflection

Rendering
2-SIDE
S-MAP
G-SHAD
S-SHAD

Edge (Outline)
On
Color
Size

Texture/Memo
Effects
Texture
Toon
SPH
Memo: AGAMENOM

Custom Effects Settings
Specular Intensity
SPH Opacity
Shadow Luminescence
HDR
Toon Tone
Multiple Lights
Fog

Surface Options
Surface Type
Render Face
Depth Write
Depth Test
Alpha Clipping
Cast Shadows
Receive Shadows

Advanced Options
Render Queue
Enable GPU Instancing
Double Sided Global Illumination
Global Illumination

Add Sphere

Add Sphere

AssetBundle
None
None

AssetBundle
None
None

Shader Properties

==System Properties==

- **Show Default Systems** = Shows default or custom properties.
- **Mat-Name (JP)** = Material name in Japanese.
- **Mat-Name (EN)** = Material name in English.
- **Memo** = Text header for notes.

==Material Color==

- **Diffuse** = Color of the material.
- **_Color ("Diffuse", Color)** : Sets the main color of the material when illuminated directly.
- **Specular** = Color of light reflection/shine.
- **_Specular ("Specular", Color)** : Sets the color of the specular reflections in the material.
- **Ambient** = Affects the color and lighting of the model, similar to a raycast.
- **_Ambient ("Ambient", Color)** : Sets the color to ambient light, contributing to the overall lighting.
- **Opaque** = Alpha value (opacity)
- **_Opaque ("Opaque", Range(0, 1))** : Controls the opacity of the material.
- **Reflection** = Reflection value
- **_Shininess ("Reflection", Float)** : Sets the intensity of the specular shine (reflection) of the material.

==Rendering==

- **2-SIDE** = Renders both sides of the mesh **[Equivalent to Render Face/_Cull]**
- **_Cull("Render Face", Float)** : Controls which faces of the mesh (front, back or both) should be rendered.
- **G-SHAD** = Shadow on the ground **[Equivalent to Cast Shadows/ShaderPass"SHADOWCASTER"]**
- **_CastShadows ("Cast Shadows", Float)** : Defines whether the material casts shadows on the ground.
- **S-MAP** = Shadow on mesh (including the mesh itself) **[Equivalent to Receive Shadows/_ReceiveShadows/Keyword"_RECEIVE_SHADOWS_OFF"]**
- **_ReceiveShadows ("Receive Shadows", Keyword)** : Controls whether the mesh receives shadows from other sources.
- **S-SHAD** = Receives shadow only from itself
- **_SShad ("S-SHAD", Float)** : Controls the shadow the mesh casts on itself only.

==Edge (Outline)==

- **On** = Activates the contour
- **_On ("On", Float)** : Controls whether the outline (border) is activated.
- **Color** = Outline color, including transparencies
- **_OutlineColor ("Color", Color)** : Sets the color of the outline around the mesh.
- **Size** = Size/distance of the contour
- **_EdgeSize ("Size", Float)** : Controls the thickness of the outline.

==Texture/Memo==

- **Texture** = Texture of the material
- **_MainTex ("Texture", 2D)** : Main texture applied to the material.
- **Toon** = Complements the object's shading
- **_ToonTex ("Toon", 2D)** : Toon texture, used for cartoon-style shading.
- **UV Layer** = UV layer to be used
- **_UVLayer ("UV Layer", Float)** : Selects which UV layer will be applied to the texture.
- **SPH** = Artificial reflection to complement Specular
- **_SphereCube ("SPH", CUBE)** : Cubic texture used for artificial reflections.

==Effects==

- **Disabled** = Does nothing
- **_EFFECTS ("Effects", Float)** : Disables effects.
- **Multi-Sphere** = Multiplies a shiny sphere map (metallic reflection)
- **_EFFECTS ("Effects", Float)** : Activates the effect of multiple glowing spheres.
- **Add-Sphere** = Creates a glowing sphere map (practical reflection)
- **_EFFECTS ("Effects", Float)** : Activates the effect of adding glowing spheres.
- **Sub-Tex** = Adds an extra UV texture layer for more complex effects
- **_EFFECTS ("Effects", Float)** : Switches from using the cubic texture to using a subtexture, applying it to another UV layer.

==Custom Effects Settings==

- **Specular Intensity** = Sets the intensity of the specular gloss
- **SpecularIntensity ("Specular Intensity", Range(0, 1))** : Intensity of the specular brightness.
- **SPH Opacity** = Sets the opacity of the Cubic Texture
- **SPHOpacity ("SPH Opacity", Range(0, 1))** : Opacity of the cubic texture.
- **Shadow Luminescence** = Shadow intensity
- **_ShadowLum ("Shadow Luminescence", Range(0, 10))** : Luminescence of shadows.
- **HDR** = Makes the object glow in the dark
- **_HDR ("HDR", Range(1, 1000))** : Controls HDR mapping.
- **Toon Tone** = Adjusts the tone of the Toon shadow
- **_ToonTone ("Toon Tone", Vector)** : Defines the tone of the shading toon.
- **Multiple Lights** = Allows the object to receive multiple lights
- **_MultipleLights ("Multiple Lights", Float)** : Enables or disables support for multiple lights.
- **Fog** = Turns fog on and off
- **_Fog ("Fog", Float)** : Enables or disables fog support.

" **Surface Options**" and "**Advanced Options**" are Unity standards that require prior knowledge of their functionality.

Known bugs

Decal not showing up in Build: To resolve this, add the Decal module to all 'Universal Renderer Data' in your project.

Distorted shadows when using Morph: To resolve this, enable the 'Legacy Blend Shape Normals'

option in the **.FBX** (or 3D model) when importing from Unity.

Light passing through objects: Add a 'Directional Light' with at least 0.001 Intensity to update the Dynamic Lightmap

Script Explanations

Copy Rotation

Overview

This component is a runtime bone rotation synchronizer designed for animation systems.

Its purpose is very simple:

- Read the rotation from one list of bones.
- Apply those same rotations to another list of bones.
- Repeat this every frame during the final animation stage.

It is commonly used for:

- Animation retargeting.
- Mirroring animation rigs.
- Synchronizing accessory skeletons.
- Driving secondary characters from a main skeleton.
- Runtime pose transfer between models.

The system only copies rotation.

It does **not**:

- Copy position.
- Copy scale.
- Smooth movement.
- Interpolate values.
- Clamp angles.
- Apply constraints.

The copied rotation is applied directly and immediately.

General Workflow

The component works in the following order every frame:

1. Check if copying is enabled.
2. Verify the bone lists are valid.
3. Verify both lists have the same amount of elements.
4. Loop through each bone pair.
5. Copy the source bone world rotation into the target bone.

Inspector Fields

These are the values exposed in the Unity Inspector.

isCopyEnabled

Controls whether the system is active.

When enabled:

- Rotations are copied every frame.

When disabled:

- The component does nothing.

This allows runtime control without removing the component.

Usage Example

Useful for:

- Temporarily disabling synchronization.
- Pausing animation transfer.
- Switching between animation systems.
- Debugging.

copyBones

This is the source bone list.

The component reads rotations from these transforms.

Each element represents a source bone.

Example:

Index	Source Bone
0	Head
1	Left Arm
2	Right Arm

The order is extremely important.

pasteBones

This is the target bone list.

The component writes rotations into these transforms.

Each element receives the rotation from the matching source index.

Example:

Index	Target Bone
0	Head Target
1	Left Arm Target
2	Right Arm Target

Important Bone Matching Rule

Both arrays must:

- Exist.
- Have the same size.
- Follow identical ordering.

Example:

Source	Target
Head	Head
Spine	Spine
Left Arm	Left Arm

If the order is wrong, incorrect rotations will be copied.

Example of bad setup:

Source	Target
Head	Left Leg
Spine	Right Arm

This would produce broken animation behavior.

Public Properties

The component exposes runtime-accessible properties.

These allow other systems to:

- Read values.
- Replace arrays.
- Enable or disable copying dynamically.

IsCopyEnabled

Provides runtime access to the enable state.

Possible uses:

- Toggle synchronization during gameplay.
- Disable copying during cutscenes.
- Activate only under certain conditions.

CopyBones

Allows runtime replacement of the source bone list.

Possible uses:

- Swap animation rigs.
- Dynamically assign characters.
- Change retargeting sources.

PasteBones

Allows runtime replacement of the target bone list.

Possible uses:

- Redirect copied animation.
- Reassign destination skeletons.
- Support modular characters.

LateUpdate Method

This is the core of the entire system.

It runs once every frame during Unity's LateUpdate phase.

Why LateUpdate Is Used

LateUpdate happens after standard animation updates.

This is important because:

- Animators usually modify bones earlier in the frame.
- The component waits until animation finishes.
- Final bone rotations are then copied safely.

Without this timing:

- Rotations could be overwritten by animation systems.
- Results could become unstable.

Using LateUpdate ensures the copied values represent the final animated pose.

Step-by-Step Execution

1. Enabled Check

The first step verifies if copying is active.

If disabled:

- The method exits immediately.
- No processing occurs.

This avoids unnecessary work.

2. Null Array Validation

The system checks whether the bone arrays exist.

Problems prevented:

- Missing assignments.
- Unassigned Inspector references.
- Runtime exceptions.

If an array is missing:

- A warning appears in the Unity Console.
- Processing stops safely.

3. Array Length Validation

The component checks whether both arrays have the same number of elements.

This is critical because:

- Each source bone needs one matching target bone.

If lengths differ:

- An error is logged.
- Copying stops.

This prevents invalid index access.

4. Bone Iteration

The component loops through all bone pairs.

Each iteration processes one pair:

- One source bone.
- One target bone.

5. Individual Bone Validation

Before copying:

- Each transform reference is checked.

If a bone is missing:

- A warning is displayed.
- That pair is skipped.
- The rest continue processing.

This prevents crashes caused by destroyed or missing transforms.

6. Rotation Copy

The final step applies the rotation.

The target bone receives the exact world rotation from the source bone.

This means:

- Orientation becomes identical.
- Rotation is copied in world space.
- Parent hierarchy differences may affect final visual results.

World Rotation Behavior

The system copies **world rotation**, not local rotation.

This means the copied orientation is based on global scene space.

Important implication:

Different parent hierarchies can produce unexpected poses.

Example:

- Source arm parent rotated 0°.
- Target arm parent rotated 90°.

Even with identical world rotation:

- Visual local pose may differ.

Performance Characteristics

This component is lightweight.

Operations performed:

- Array validation.
- Simple loop iteration.
- Direct rotation assignment.

Performance cost mainly depends on:

- Bone count.
- Number of active components.

Typical humanoid rigs are usually inexpensive.

Common Use Cases

Animation Retargeting

Transfer animation between different skeletons.

Runtime Pose Mirroring

Drive one character from another character's movement.

VR Tracking Systems

Apply tracked bone rotations onto avatars.

MMD / VTuber Systems

Synchronize imported animation rigs.

Accessory Skeleton Sync

Make clothing or armor rigs follow a character skeleton.

Limitations

This system intentionally keeps behavior simple.

It does not solve:

- Bone hierarchy differences.
- Scale compensation.
- Offset correction.
- Rotation blending.
- IK stabilization.

- Axis remapping.

Those features would require additional processing layers.

Important Setup Recommendations

For best results:

- Use matching bone hierarchies.
- Maintain identical bone ordering.
- Ensure transforms are assigned correctly.
- Prefer similar rig orientations.
- Avoid mismatched parent rotations when possible.

Summary

This component is a minimal runtime rotation transfer system.

Its design focuses on:

- Simplicity.
- Predictability.
- Low overhead.
- Direct world-space synchronization.

Every frame:

- Source bone rotations are read.
- Target bone rotations are overwritten.
- The target skeleton mirrors the source orientation exactly.

Realtime Lip Sync Analyzer

Overview

This system is a real-time voice analysis and lip sync controller designed for stylized character animation systems such as:

- MMD.
- VRM avatars.
- VTubers.
- Anime-style facial rigs.
- Runtime talking characters.

The component listens to live audio, analyzes the frequency content of the voice, estimates which vowel is being spoken, and converts that information into facial mouth blend shape animation.

The system is inspired by the classic Japanese MMD tool:

- LipSynchroid

Instead of attempting perfect speech recognition, the system focuses on:

- Stable animation.
- Responsive mouth movement.
- Reduced jitter.
- Real-time usability.

The system specifically targets Japanese vowel morph systems:

Vowel	Japanese
A	あ
I	い
U	う
O	お

General Runtime Flow

Every frame the system performs the following sequence:

1. Read audio data from the audio source.
2. Measure overall loudness.
3. Extract FFT frequency spectrum.
4. Analyze vowel frequency regions.
5. Estimate which vowel is dominant.
6. Convert the vowel into morph weights.
7. Scale intensity using voice volume.

8. Smooth and stabilize values.
9. Apply final blend shape weights.

Main Purpose of the System

The component transforms sound into facial animation.

More specifically:

Audio Input	Output
Voice frequencies	Mouth shape selection
Voice loudness	Mouth opening intensity
Frequency patterns	Vowel estimation
Real-time speech	Blend shape animation

Serialized Fields

These are the settings visible in the Unity Inspector.

References Section

audioSource

This is the voice source used for analysis.

The system reads:

- Raw waveform samples.
- Frequency spectrum information.

Without this:

- No audio can be analyzed.
- Lip sync cannot function.

Usage

Usually connected to:

- Microphone input.
- Voice playback.
- Streaming audio.
- Dialogue systems.
- Live VTuber microphones.

skinnedMeshRenderer

This is the facial mesh containing the mouth blend shapes.

The system writes morph values directly into this mesh.

Without this:

- Audio analysis still occurs.
- But no visible facial animation is produced.

Usage

Usually connected to:

- Character face mesh.
- Avatar head mesh.
- VRM facial renderer.
- MMD face model.

Blend Shape Indices

These variables define which morph corresponds to each vowel.

vowelABlendShapeIndex

Defines the mouth morph used for vowel A.

Usually:

- Wide open mouth.
- Large jaw opening.

Japanese:

- あ

vowelIBlendShapeIndex

Defines the mouth morph used for vowel I.

Usually:

- Horizontal stretched mouth.

Japanese:

- い

vowelUBlendShapeIndex

Defines the mouth morph used for vowel U.

Usually:

- Rounded narrow lips.

Japanese:

- う

vowelOBBlendShapeIndex

Defines the mouth morph used for vowel O.

Usually:

- Circular open mouth.

Japanese:

- お

Dynamic Range Settings

These settings control voice sensitivity.

minimumDecibelThreshold

Defines the minimum volume required before the mouth starts moving.

Sounds below this threshold are treated as silence.

Purpose

Prevents:

- Noise triggering.
- Breathing animation.
- Microphone hiss.
- Tiny mouth jitter.

maximumDecibelThreshold

Defines the volume where mouth intensity reaches maximum strength.

Anything louder than this:

- Is clamped to full morph intensity.

Purpose

Prevents:

- Excessive mouth deformation.
- Unstable exaggerated morphs.

Temporal Smoothing Settings

These settings stabilize animation.

smoothingWindowSize

Controls how many historical samples are used for smoothing.

Higher values:

- Smoother animation.
- Less jitter.
- More latency.

Lower values:

- Faster response.
- Sharper movement.
- More instability.

interpolationSpeed

Controls blend shape transition speed.

Lower values:

- Smooth slow movement.
- Higher values:
- Faster snappier motion.

stabilizationDeadzone

Defines how much change is required before animation reacts strongly.

Tiny changes below this threshold:

- Are softened.
- Partially ignored.

Purpose

Reduces:

- Facial twitching.
- Rapid micro fluctuations.
- Noise instability.

Runtime Buffers

These are internal working memory buffers.

audioSamples

Stores raw waveform audio samples.

Used for:

- Loudness measurement.
- RMS calculation.

frequencySpectrum

Stores FFT spectrum data.

This contains:

- Frequency energy distribution.

Used for:

- Vowel estimation.
- Formant analysis.

currentBlendShapeWeights

Stores the current active mouth weights.

The system continuously updates these values.

vowelSmoothingFilters

Contains temporal smoothing filters.

Each vowel has its own filter:

- A.
- I.
- U.
- O.

These filters stabilize motion over time.

Vowel Weight Table

VowelWeightTable

This is the core morph behavior table.

Each row defines how strongly each vowel affects the blend shapes.

Example concept:

Detected Vowel	A	I	U	O
A	Strong	None	None	None
U	Partial	None	Strong	Small
Neutral	Mixed	Mixed	Mixed	Small

This creates:

- More natural transitions.
- Softer phoneme blending.
- Less robotic movement.

Start Method

This initializes the smoothing filters.

One filter is created for each vowel channel.

Without initialization:

- Temporal smoothing would not work.

LateUpdate Method

This is the heart of the entire system.

It runs every frame after animation updates.

Why LateUpdate Is Important

Lip sync should happen after:

- Animator updates.
- Facial rig updates.
- Other animation systems.

This ensures:

- Final mouth values are applied last.
- Animation conflicts are minimized.

Step-by-Step Runtime Analysis

Step 1 — Capture Audio Samples

The system reads live audio waveform data.

Then it calculates:

- RMS loudness.

RMS represents:

- Average sound intensity.

This becomes:

- Voice volume estimation.

Root Mean Square (RMS)

RMS converts fluctuating waveform samples into a stable loudness value.

Without RMS:

- Loudness would fluctuate violently.

Decibel Conversion

The RMS value is converted into decibels.

This creates:

- Human-like loudness scaling.

Used for:

- Mouth opening intensity.

Step 2 — FFT Spectrum Extraction

The system performs FFT analysis.

FFT converts:

- Time-domain audio
into:
- Frequency-domain data.

Meaning:

Instead of seeing:

- Wave amplitude over time.

The system sees:

- Energy at different frequencies.

Why FFT Matters

Different vowels contain different frequency patterns.

Example:

Vowel

Frequency Characteristics

A	Strong lower frequencies
I	Strong mid-high frequencies
U	Narrow concentrated bands
O	Rounded lower-mid formants

FFT makes vowel estimation possible.

Step 3 — Formant Region Analysis

The system analyzes specific frequency ranges.

Each range represents a possible vowel formant region.

Example:

Vowel	Frequency Band
A	0–1300 Hz
I	800–1400 Hz
U	1500–2000 Hz
O	2800–3200 Hz

The system measures:

- How much energy exists in each band.

Normalized Band Energy

The energy is normalized against total spectrum energy.

Meaning:

- Relative vowel dominance is measured.

This avoids:

- Loudness bias.
- Raw amplitude dependency.

Step 4 — Detection Flags

The system converts frequency analysis into binary flags.

Each vowel region can become:

- Active.
- Inactive.

These flags create a compact vowel signature.

Step 5 — Vowel Classification

The detection flags are analyzed using heuristic rules.

The system estimates:

- Which vowel is most likely dominant.

This is NOT true speech recognition.

Instead:

- It is pattern matching based on spectral shapes.

The result is:

- Fast.
- Lightweight.
- Stable.

Perfect for real-time avatars.

Step 6 — Morph Weight Retrieval

After vowel classification:

- The corresponding morph weight profile is selected.

The selected profile determines:

- Which mouth shapes activate.
- Their relative strengths.

Step 7 — Amplitude Scaling

The mouth intensity is scaled according to voice loudness.

Quiet voice:

- Small mouth movement.

Loud voice:

- Large mouth movement.
- This produces:
- Natural intensity variation.

Step 8 — Smoothing and Stabilization

This is one of the most important parts.
Raw audio analysis is naturally unstable.
Without smoothing:

- Mouth movement becomes chaotic.
- Blend shapes flicker constantly.

The system solves this using:

- Weighted temporal filtering.
- Deadzone stabilization.
- Interpolation.

WeightedHistoryFilter

This internal filter smooths values over time.
Recent samples receive higher importance.
Older samples gradually lose influence.

This creates:

- Responsive motion.
- Stable animation.
- Reduced jitter.

Exponential Weighting

Recent history has stronger influence than old history.

Example:

Sample Age	Influence
Newest	Very high
Older	Reduced
Oldest	Minimal

This avoids:

- Excessive lag.
- Delayed mouth response.

Deadzone Stabilization

Tiny fluctuations are partially ignored.

This prevents:

- Constant micro mouth movement.
- Noise twitching.

Especially important for:

- Microphones.
- Live streaming.
- Low-quality audio.

Interpolation

The system gradually transitions toward target values.

Instead of:

- Instantly snapping.

It:

- Smoothly moves.

This creates:

- Organic animation.
- Softer speech movement.

Silence Handling

When silence is detected:

- Filters gradually decay toward zero.

This creates:

- Smooth mouth closing.
- Natural relaxation.

Instead of:

- Instant shutdown.

ResetBlendShapes Method

This method softly returns the mouth to neutral.

Used when:

- Audio stops.
- Silence occurs.
- Invalid spectrum data appears.

The reset is gradual to avoid:

- Visual popping.
- Abrupt facial changes.

Audio Analysis Methods

CalculateRootMeanSquare

Measures overall waveform loudness.

Purpose:

- Stable amplitude estimation.

CalculateNormalizedBandEnergy

Measures energy inside a specific frequency range.

Purpose:

- Detect vowel formants.

Vowel Classification Method

ClassifyVowel

Uses heuristic rules to estimate the active vowel.

This is essentially:

- A lightweight phoneme classifier.

Not AI.

Not machine learning.

Pure rule-based spectral analysis.

Advantages:

- Fast.
- Deterministic.
- Low CPU usage.

Morph Processing Methods

CalculateAmplitudeScaling

Converts decibel volume into normalized morph intensity.

Result range:

- 0 to 1.

Used for:

- Mouth opening strength.

ApplyBlendShapeWeights

This is where final facial animation occurs.

The method:

1. Applies amplitude scaling.
2. Applies smoothing filters.
3. Applies stabilization.
4. Interpolates motion.
5. Writes final blend shape values.

Internal WeightedHistoryFilter Class

This is the temporal smoothing engine.

It stores:

- Recent historical values.

Then calculates:

- Weighted smoothed output.

Purpose:

- Eliminate unstable motion.
- Preserve responsiveness.

Editor Tools

The script also includes a custom Unity Editor extension.

This only exists inside the Unity Editor.

It does not run in builds.

Debug Graph System

The inspector displays real-time vowel graphs.

Each curve represents:

- One vowel channel.

Colors:

- Red = A.
- Blue = I.
- Green = U.
- Yellow = O.

This allows:

- Real-time debugging.
- Lip sync tuning.
- Smoothing visualization.

Auto Detect Blend Shapes

The editor can automatically search for vowel morphs.

It scans:

- Blend shape names.

Supports:

- Japanese names.
- English names.
- Common VRM/MMD naming conventions.

Flexible Name Matching

The detection system handles names like:

- A.
- mouth_a.
- face-a.
- あ.

This improves compatibility across:

- MMD models.
- VRM avatars.
- Custom rigs.

Typical Usage Workflow

1. Assign Audio Source

Connect:

- Microphone.
- Voice playback.
- Dialogue system.

2. Assign Face Mesh

Connect:

- Facial skinned mesh renderer.

3. Configure Blend Shapes

Assign:

- A morph.
- I morph.
- U morph.
- O morph.

Or use:

- Auto Detect.

4. Tune Thresholds

Adjust:

- Minimum dB.
- Maximum dB.
- Smoothing.
- Stabilization.

Until motion feels natural.

5. Run in Real Time

The system automatically:

- Detects vowels.
- Animates the mouth.
- Stabilizes speech motion.

Main Design Philosophy

This system prioritizes:

- Real-time stability.
- Lightweight execution.
- Stylized animation quality.
- Smooth avatar motion.

Over:

- Linguistic precision.
- True phoneme recognition.
- Speech transcription.

It is designed for:

- Visual believability.
- Not scientific speech analysis.

Summary

This component is a real-time vowel-driven facial animation system.

It transforms:

- Voice frequency patterns into:
- Stable mouth animation.

Internally it combines:

- FFT spectrum analysis.
- Formant energy detection.
- Heuristic vowel classification.
- Dynamic amplitude scaling.
- Temporal smoothing.
- Blend shape interpolation.
- Stabilization filtering.

The result is:

- Lightweight.
- Responsive.
- Stable.
- MMD-style lip sync animation suitable for games, VTubers, and live avatars.

Lip Sync Recorder

Overview

This system is a real-time lip sync recorder and animation baker.

Its purpose is to:

1. Read the live mouth animation generated by the lip sync analyzer.
2. Record the blend shape values over time.
3. Convert the captured motion into a permanent animation clip.
4. Save the animation as a reusable .anim asset.

In practice, the component transforms:

Real-Time Lip Sync	Saved Animation
Live mouth movement	Baked keyframes
Runtime blend shapes	Animation curves
Temporary speech animation	Reusable animation asset

This allows developers to:

- Convert procedural lip sync into editable animation.
- Export mouth motion into reusable clips.
- Reduce runtime CPU cost by baking.
- Fine-tune lip sync manually afterward.

Main Purpose of the System

The recorder acts like a motion capture system for facial blend shapes.

Instead of recording:

- Body motion.

It records:

- Mouth morph values.

More specifically:

- A.
- I.
- U.
- O vowel weights.

The system captures these values frame-by-frame and reconstructs them into animation curves.

High-Level Workflow

The complete process works like this:

1. Prepare the audio.
2. Wait through a stabilization delay.
3. Start audio playback.
4. Record blend shape values every sample frame.
5. Store recorded keyframes.
6. Stop when audio ends.
7. Build animation curves.
8. Optimize curves.
9. Save the animation clip.

Recording States

The recorder operates as a state machine.

RecordingState

This defines the current stage of the recording process.

Idle

The recorder is inactive.

Nothing is:

- Recording.
- Sampling.
- Saving.

This is the default state.

PreRoll

The recorder is waiting before recording begins.

Purpose:

- Allow stabilization.
- Let filters settle.
- Prevent abrupt mouth starts.

This creates smoother recording startup.

Recording

The recorder is actively capturing blend shape values.

During this state:

- Samples are stored repeatedly over time.

Saving

Recording has finished.

The system is now:

- Building animation curves.
- Optimizing data.
- Saving the final clip.

Serialized Fields

These settings appear in the Unity Inspector.

Avatar Settings

targetAvatar

Defines the root avatar object.

This is used to calculate:

- Relative animation paths.

Animation clips require:

- Correct hierarchy paths to know which object receives animation.

Without this:

- The generated animation may target the wrong transform.

Recording Settings

preRollDelay

Defines how long the system waits before recording begins.

Purpose:

- Stabilize lip sync filters.
- Allow audio systems to settle.
- Prevent harsh animation startup.

Example:

- 2 seconds.
- 5 seconds.
- 10 seconds.

During this time:

- Nothing is recorded yet.

sampleRate

Defines how many samples are captured per second.

Equivalent to:

- Recording FPS.

Example:

Sample Rate	Result
15	Low detail
30	Standard
60	Very smooth

Higher values:

- More accurate animation.
- Larger file size.
- More keyframes.

Lower values:

- Smaller files.
- Less precision.

optimize

Controls whether curve optimization is enabled.

When enabled:

- Redundant keyframes are removed.

Benefits:

- Smaller animation files.
- Cleaner curves.
- Better performance.

Without optimization:

- Every recorded frame remains.

Runtime Buffers

These are temporary recording structures.

state

Stores the current recording state.

Used by:

- Editor UI.
- Recording logic.
- Progress tracking.

analyzer

Cached reference to the lip sync analyzer.

The recorder reads:

- Blend shape values from this analyzer.

Without it:

- Nothing can be recorded.

timer

General timing counter.

Used for:

- Pre-roll timing.
- Recording duration.
- Sample scheduling.

nextSampleTime

Tracks when the next sample should be captured.

This ensures:

- Stable sample intervals.
- Consistent frame timing.

Keyframe Buffers

keyA

Stores recorded keyframes for vowel A.

keyI

Stores recorded keyframes for vowel I.

keyU

Stores recorded keyframes for vowel U.

keyO Stores recorded keyframes for vowel O.
Public Properties These allow runtime access to settings.
TargetAvatar Provides runtime access to the avatar root.
PreRollDelay Allows runtime modification of stabilization delay.
SampleRate Allows runtime modification of recording FPS.
Awake Method This initializes internal references. The recorder automatically locates: - The lip sync analyzer attached to the same object. This avoids: - Manual assignment.
Update Method This is the heart of the recorder. It runs every frame and manages: - State transitions. - Sampling. - Recording timing. - Saving logic.
Update Flow The update loop behaves differently depending on the current state.
Idle State If the recorder is idle: The update exits immediately. No work is performed.
PreRoll State The recorder waits until: - The stabilization delay expires. Once complete: - Audio playback starts. - Recording begins.
Recording State During recording: - Blend shape values are sampled repeatedly. The recorder checks: - Has the next sample time been reached? If yes: - A frame is captured.
Audio Completion Detection The recorder continuously checks: - Whether the audio has finished playing. When playback ends: - Final frame is captured. - Recording stops.

3. Baking begins.
4. Animation is saved.
5. State returns to idle.

Core Recording Logic

StartRecording Method

This begins the recording session.
The method performs several preparation steps.

Audio Reset

The audio source is reset to:

- Time zero.

This guarantees:

- Perfect synchronization.

Loop Disabling

Looping is disabled.

Purpose:

- Prevent endless recording.

Keyframe Cleanup

All previous recording data is cleared.

This ensures:

- Clean recording sessions.

Timer Reset

Recording timers are reset.

Necessary for:

- Accurate synchronization.

PreRoll Activation

The system enters:

- Pre-roll state.

Recording does not start immediately.

StartAudioAndRecording Method

This begins actual recording.

The method:

1. Restarts audio playback.
2. Resets timing.
3. Switches into recording state.

Now:

- Sampling begins.

CaptureFrame Method

This is the actual sampling function.

Every time it runs:

- Current blend shape weights are recorded.

The recorder stores:

Channel	Recorded Data
A	Current weight
I	Current weight
U	Current weight
O	Current weight

Each sample becomes:

- A keyframe.

What a Keyframe Represents

Each keyframe stores:

- Time.
- Blend shape value.

Example:

Time	Weight
0.0	0
0.1	25
0.2	60

Over time:

- These form animation curves.

BakeClip Method

This is the most important method in the entire system.
It converts raw recorded data into a real animation clip.

Step 1 — Validation

The method first checks:

- Avatar exists.
- Renderer exists.

Without them:

- Animation cannot be created.

Step 2 — Relative Path Calculation

The system calculates:

- The transform path from avatar root to face mesh.

Animation clips require this path to:

- Know which object receives animation.

Step 3 — Animation Clip Creation

A new animation clip is created.

The clip uses:

- The configured sample rate.

Step 4 — Curve Construction

The recorded keyframes are converted into:

- Animation curves.

Each vowel receives its own curve.

Step 5 — Curve Optimization

If optimization is enabled:

- Curves are simplified.

This removes:

- Redundant data.
- Flat unnecessary frames.

Result:

- Smaller animation assets.
- Cleaner curves.

Step 6 — Blend Shape Binding

Each curve is linked to:

- A specific blend shape.

This tells Unity:

- Which mouth morph each curve controls.

Step 7 — Save Path Creation

The output path is generated automatically.

The animation is saved:

- Next to the source audio clip.

Example:

Audio File	Generated Animation
Voice.wav	Voice_LipSync.anim
Step 8 — Existing File Replacement If a previous animation already exists: - It is deleted first. This prevents: - Duplicate assets. - Conflicts.	
Step 9 — Save Asset The animation asset is created and saved. At this point: The baked lip sync animation becomes permanent.	
Step 10 — Stop Play Mode After baking: Play mode automatically exits. This streamlines editor workflow.	
Curve Optimization System This is one of the most advanced parts of the script.	
Why Optimization Exists Raw recording creates: - Massive numbers of keyframes. Many are unnecessary because: - Mouth movement often remains stable. Optimization reduces: - File size. - Memory usage. - Curve complexity.	
OptimizeCurve Method This method simplifies animation curves while preserving visible motion.	
Stage 1 — Flat Region Removal Tiny changes are detected. If movement is nearly identical: - Intermediate keys are removed. This eliminates: - Noise. - Redundant frames.	
Stage 2 — Peak Preservation Important peaks and valleys are preserved. Purpose: - Keep expressive motion. Without this: - Mouth shapes could lose definition.	
Stage 3 — Zero Preservation Exact zero values are protected. This is important because: Mouth closure is visually critical.	
Stage 4 — Ramer-Douglas-Peucker Simplification The system uses a geometric simplification algorithm. Purpose:	

- Preserve overall curve shape while reducing points.

The algorithm:

- Detects which keys meaningfully affect the curve.
- Removes the rest.

Why RDP Is Important

Without it:

- Curves remain overly dense.

With it:

- Animation becomes dramatically cleaner.

Tangent Smoothing

After optimization:

- Automatic smooth tangents are applied.

This creates:

- Natural curve interpolation.
- Softer transitions.

GetPerpendicularDistance Method

This supports the RDP simplification algorithm.

Purpose:

- Measure how far a keyframe deviates from a straight line.

Keys with minimal deviation:

- Are considered unnecessary.

Progress API

GetProgress Method

Returns normalized recording progress.

Range:

- 0 to 1.

Used by:

- Editor UI.
- Progress bars.
- Visual feedback.

Progress Behavior

State	Progress Meaning
PreRoll	Stabilization progress
Recording	Audio playback progress
Saving	Fully complete
Idle	No progress

Editor Integration

The script includes a custom Unity Editor extension.

This creates:

- Recording controls.
- Progress visualization.
- Real-time workflow tools.

Custom Inspector Features

Play / Record Button

The inspector provides:

- A single recording button.

Behavior changes automatically depending on:

- Whether Unity is already in play mode.

Progress Bar

The inspector shows:

- Live recording progress.

Colors indicate state:

Color	Meaning
Red	Stabilizing
Cyan	Recording
Green	Saving

Real-Time Repainting

The inspector refreshes continuously during recording.

This keeps:

- Progress feedback live.

Typical Workflow

1. Configure Lip Sync Analyzer

Set up:

- Audio source.
- Blend shapes.
- Lip sync system.

2. Assign Target Avatar

Specify:

- The avatar root object.

3. Enter Play Mode

The system prepares runtime recording.

4. Start Recording

The recorder:

- Waits through pre-roll.
- Starts audio playback.
- Begins sampling.

5. Record Blend Shapes

Mouth animation is captured in real time.

6. Bake Animation

After audio ends:

- Curves are generated.
- Optimized.
- Saved automatically.

7. Reuse the Animation

The generated .anim file can now be:

- Played normally.
- Edited manually.
- Added to timelines.
- Used in cutscenes.
- Exported with avatars.

Main Design Philosophy

This system focuses on:

- Converting procedural lip sync into reusable assets.
- Automating mouth animation baking.
- Reducing manual animation work.
- Preserving stable facial motion.

It prioritizes:

- Workflow speed.

- Animation cleanliness.
- Editor usability.

Over:

- Runtime-only behavior.

Summary

This component is a real-time lip sync animation recorder and baking system.

It works by:

1. Monitoring live blend shape values.
2. Recording them over time.
3. Converting them into animation curves.
4. Optimizing the curves.
5. Saving a reusable animation clip.

Internally it combines:

- Real-time sampling.
- Temporal synchronization.
- Keyframe recording.
- Curve generation.
- Redundant key removal.
- Geometric curve simplification.
- Automatic asset creation.

The final result is:

- A clean baked lip sync animation generated automatically from real-time procedural speech motion.

Free Fly Camera

Overview

This component is a free-fly camera controller designed for unrestricted scene navigation.

It behaves similarly to:

- Editor fly cameras.
- First-person debug cameras.
- Spectator cameras.
- Cinematic navigation tools.

The system allows the camera to:

- Move freely in 3D space.
- Rotate using mouse movement.
- Zoom using the scroll wheel.
- Switch between slow and fast traversal speeds.

Unlike character controllers:

- It is not physics-based.
- It does not use gravity.
- It ignores collisions.
- It has no movement constraints.

The camera behaves like a fully detached observer.

Main Purpose of the System

The component provides a lightweight navigation camera for:

- Scene inspection.
- Debugging.
- Level design.
- Cinematic previews.
- Runtime editor tools.
- Sandbox exploration.

It is especially useful when developers need:

- Fast traversal.
- Precision movement.
- Direct camera control.

General Workflow

Every frame the system performs the following sequence:

1. Detect input keys.
2. Determine movement speed.
3. Build movement direction.
4. Move the camera.
5. Detect look mode state.
6. Rotate camera from mouse movement.
7. Process zoom input.
8. Handle cursor locking/unlocking.

Camera Settings

These settings control movement and camera responsiveness.

movementSpeed

Defines the normal movement speed.

Used when:

- Fast mode is NOT active.

This controls:

- Standard navigation speed.

Lower values:

- Slower precise movement.

Higher values:

- Faster traversal.

fastMovementSpeed

Defines the movement speed while holding the run key.

Purpose:

- Rapid movement through large scenes.

Useful for:

- Open worlds.
- Large environments.
- Fast debugging traversal.

sensitivity

Controls mouse rotation sensitivity.

This affects:

- How quickly the camera rotates while looking.

Lower values:

- Smooth precise rotation.

Higher values:

- Faster aggressive turning.

zoomSensitivity

Defines normal zoom speed.

Used when:

- Scroll wheel is used normally.

Controls:

- Forward/backward zoom distance.

fastZoomSensitivity

Defines zoom speed while fast mode is active.

Purpose:

- Rapid camera repositioning.

Especially useful in:

- Very large scenes.
- Cinematic inspection.

Key Settings

These variables define all movement controls.
The system supports full customization.

Local Space Movement

These controls move relative to the camera orientation.

Meaning:

- Movement direction changes depending on where the camera is facing.

left

Moves camera left relative to current rotation.

Default:

- A key.

right

Moves camera right relative to current rotation.

Default:

- D key.

front

Moves camera forward relative to viewing direction.

Default:

- W key.

back

Moves camera backward relative to viewing direction.

Default:

- S key.

Local Vertical Movement

These use the camera's own up direction.

Meaning:

- Movement follows camera tilt.

up

Moves upward relative to camera orientation.

Default:

- Q key.

down

Moves downward relative to camera orientation.

Default:

- E key.

Global Vertical Movement

These use world-space vertical direction.

Meaning:

- Always aligned to the world's vertical axis.

Independent from camera rotation.

upGlobal

Moves upward using global world up.

Default:

- R key.

Useful when:

- Camera is tilted sideways.
- You still want true vertical motion.

downGlobal

Moves downward using world-space down.

Default: • F key.
Look Mode Control
observe Defines the key that activates look mode. Default: • Right Mouse Button. While held: • Mouse movement rotates the camera.
Fast Mode Control
run Defines the fast movement modifier key. Default: • Left Shift. While held: • Movement speed increases. • Zoom speed increases.
Axis Settings These define the input axis names. They allow compatibility with Unity's input system.
zoom Defines the scroll wheel axis. Used for: • Forward/backward zoom movement. Default: • Mouse ScrollWheel.
axisX Defines horizontal mouse movement axis. Controls: • Horizontal camera rotation. Default: • Mouse X.
axisY Defines vertical mouse movement axis. Controls: • Vertical camera rotation. Default: • Mouse Y.
Runtime State
looking Tracks whether look mode is currently active. When true: • Mouse controls camera rotation. When false: • Mouse movement does nothing.
Public Properties The component exposes all settings through properties. This allows:

- Runtime modification.
- Tool integration.
- Dynamic camera customization.

Examples:

- Changing sensitivity dynamically.
- Switching movement profiles.
- Rebinding controls.

Update Method

This is the core of the entire camera system.

It runs every frame.

The method handles:

- Movement.
- Rotation.
- Zoom.
- Input processing.

Step-by-Step Runtime Flow

Step 1 — Fast Mode Detection

The system first checks:

- Whether the run key is being held.

If active:

- Fast movement speed is used.
- Fast zoom speed is used.

Otherwise:

- Normal speeds are used.

Step 2 — Movement Direction Construction

The system builds a direction vector.

Every pressed key contributes movement.

Example:

Key	Direction Added
W	Forward
S	Backward
A	Left
D	Right

Multiple keys combine together.

Example:

- W + D produces diagonal forward-right movement.

Local Space Movement

Most movement directions are relative to camera orientation.

Meaning:

- Forward always means "where the camera looks".

This creates:

- Natural first-person navigation.

World Space Vertical Movement

Global up/down ignores camera tilt.

This is important because:

- A tilted camera would otherwise create strange vertical motion.

The component supports both:

- Local vertical movement.
- Absolute world vertical movement.

This provides flexibility for:

- Cinematic work.
- Scene editing.
- Inspection tools.

Step 3 — Position Update

After calculating movement direction:

- Camera position is updated.

Movement depends on:

Factor	Purpose
Movement speed	Travel speed
Delta time	Frame-rate independence
Direction vector	Movement orientation

Why Delta Time Matters

Movement uses frame-independent timing.

Without this:

- Faster PCs move faster.
- Slower PCs move slower.

Using delta time ensures:

- Consistent motion across frame rates.

Step 4 — Look Mode Rotation

If look mode is active:

- Mouse movement rotates the camera.

Horizontal Rotation

Horizontal mouse movement affects:

- Yaw rotation.

Meaning:

- Looking left/right.

Vertical Rotation

Vertical mouse movement affects:

- Pitch rotation.

Meaning:

- Looking up/down.

Mouse Sensitivity

Mouse input is multiplied by sensitivity.

This controls:

- Rotation responsiveness.

Roll Removal

The system always forces roll rotation to zero.

Meaning:

- Camera never tilts sideways automatically.

This keeps:

- Stable horizon alignment.
- Easier navigation.

Step 5 — Zoom Handling

The scroll wheel moves the camera forward or backward.

Unlike changing camera FOV:

- This physically moves the camera.

Result:

- True positional zooming.

Zoom Direction

Zoom occurs along:

- Camera forward direction.

Meaning:

- Zoom follows current viewing angle.

Fast Zoom

If fast mode is active:

- Zoom becomes much stronger.

Useful for:

- Quickly approaching distant objects.

Step 6 — Look Mode Activation

The system checks:

- Whether the observe key was pressed.

If pressed:

- Look mode starts.

Step 7 — Look Mode Deactivation

The system checks:

- Whether the observe key was released.

If released:

- Look mode stops.

StartLooking Method

This enables look mode.

The method performs three actions:

1. Enables rotation mode.
2. Hides the mouse cursor.
3. Locks the cursor to the screen center.

Why Cursor Locking Matters

Without cursor locking:

- Mouse would leave the game window.
- Rotation would stop at screen edges.

Locking allows:

- Infinite continuous rotation.

Why Cursor Hiding Matters

During camera look:

- Cursor becomes distracting.

Hiding improves:

- Immersion.
- Usability.

StopLooking Method

This disables look mode.

The method:

1. Stops camera rotation input.
2. Makes cursor visible again.
3. Unlocks the cursor.

This restores:

- Normal desktop interaction.

OnDisable Method

This runs when:

- The component is disabled.
- The object is destroyed.
- Play mode exits.

Purpose:

- Ensure cursor state is restored correctly.

Without this:

- Cursor could remain hidden or locked accidentally.

Movement Behavior
No Physics The camera: • Ignores collisions. • Passes through objects. • Uses direct transform movement. Advantages: • Simplicity. • Speed. • Precision. Disadvantages: • No physical interaction.
Diagonal Movement Movement vectors combine directly. This means: • Diagonal movement is naturally supported. Example: • W + D + Q moves: ○ Forward. ○ Right. ○ Up simultaneously.
Rotation Behavior
Euler Rotation System The camera uses Euler-style rotation. Advantages: • Simple. • Intuitive. • Easy to customize. Potential limitation: • Extreme rotations may create flipping behavior. However: • For free-fly cameras this is usually acceptable.
Typical Usage Scenarios
Scene Navigation Quickly move around large environments.
Debug Cameras Inspect gameplay systems during runtime.
Cinematic Tools Position cameras manually for shots.
Model Inspection Examine characters and assets freely.
Sandbox Exploration Fly through procedural or open-world scenes.
Runtime Editors Use as a camera inside custom editors.
Common Control Scheme Typical default controls:

Action	Key
Forward	W
Backward	S
Left	A
Right	D
Up Local	Q
Down Local	E
Up Global	R
Down Global	F
Look Around	Right Mouse
Fast Mode	Shift
Zoom	Mouse Wheel

Design Philosophy

The component prioritizes:

- Fast navigation.
- Minimal overhead.
- Full movement freedom.
- Simplicity.
- Tool usability.

Over:

- Realistic physics.
- Character simulation.
- Collision systems.

Performance Characteristics

The system is extremely lightweight.

Operations performed:

- Basic input checks.
- Simple vector math.
- Direct transform updates.

CPU cost is very low.

Suitable for:

- Runtime tools.
- Editors.
- Debug builds.
- Cinematic utilities.

Summary

This component is a free-fly navigation camera system designed for unrestricted 3D scene movement.

It provides:

- Six-directional movement.
- Mouse-look rotation.
- Scroll-wheel zoom.
- Fast traversal mode.
- Local and global vertical movement.
- Cursor locking.
- Runtime customization.

Internally it combines:

- Input-driven directional movement.
- Frame-independent motion.
- Mouse-based rotation.
- Dynamic speed switching.
- Real-time camera positioning.

The final result is:

- A lightweight, flexible, editor-style fly camera suitable for debugging, scene exploration, cinematic tools, and runtime navigation systems.

Custom MMD Data

Overview

This system acts as the central database and initialization layer for the entire MMD rendering framework. Its purpose is to store shared material metadata and guarantee that the project always contains a single valid global configuration asset.

In practice, this script solves three important problems:

- Centralizing MMD material information.
- Providing global configuration flags for editor tools.
- Automatically creating the database asset if it does not exist.

The system is divided into two major parts:

1. Runtime data structures.
2. Editor-only automatic asset initialization.

Main Purpose of the System

The system is designed to work like a lightweight internal database for MMD-related tools.

Instead of every tool storing material information separately, all data is centralized into one shared asset.

This allows:

- Material localization support.
- Shader debugging notes.
- Global framework settings.
- Consistent access across tools.
- Automatic loading through the Resources system.

Core Classes

The script contains three main structures:

Structure	Purpose
CustomMMDData	Main database asset.
MMDMaterialInfo	Individual material metadata entry.
CustomMMDDDataStartup	Automatic editor initialization system.

CustomMMDDData

This is the main database container.

It exists as a project asset instead of a scene object.

That means:

- The data persists independently from scenes.
- All tools can access the same information globally.
- The configuration survives editor restarts.

You can think of it as:

"The central registry for all MMD material information."

Variables Inside CustomMMDDData

showSystemsDefault

Determines whether advanced system information should be visible by default inside editor tools.

Purpose

Some MMD editor tools may contain:

- Debug information.
- Internal shader systems.
- Advanced rendering options.

This variable controls whether those sections start expanded or hidden.

Usage Example

Useful when:

- Developers want detailed debugging visible automatically.
- Artists prefer a cleaner interface.

materialInfoList

A collection containing all registered MMD materials and their metadata.

Each element inside the list is an MMDMaterialInfo.

Purpose

Acts as the actual material database.

Stores:

- Material references.
- Japanese names.
- English names.
- Developer notes.

Usage Example

A tool could search this list to:

- Match imported MMD materials.
- Display translated names.
- Show shader compatibility notes.
- Organize rendering pipelines.

MMDMaterialInfo

Represents one single material entry inside the database.

Each object describes one material and its metadata.

Variables Inside MMDMaterialInfo

mmdMaterial

Reference to the actual Unity material.

Purpose

Links the metadata entry to a real render material.

Without this reference, the system would only contain text information.

Usage Example

Used to:

- Identify materials during import.
- Apply MMD shaders.
- Match materials automatically.
- Display material information in editor tools.

materialNameJP

Japanese display name of the material.

Purpose

Maintains compatibility with original MMD naming conventions.

Many MMD assets originate from Japanese workflows, so preserving original names is important.

Usage Example

Could be used when:

- Importing PMX models.
- Displaying original Japanese metadata.
- Preserving compatibility with Japanese MMD tools.

materialNameEN

English display name of the material.

Purpose

Provides readable localized names for international users.

Usage Example

Instead of seeing:

- 「髪影」

A tool could display:

- "Hair Shadow"

This improves usability for non-Japanese developers.

materialMeno

Developer memo field.

(Probably intended to mean "Memo".)

Purpose

Stores optional notes related to the material.

Useful for:

- Shader debugging.

- Pipeline comments.
- Technical warnings.
- Rendering observations.

Usage Example

A developer might store notes like:

- "Requires outline shader."
- "Transparency sorting issue."
- "Compatible with toon lighting only."

Editor Startup System

The second half of the script only exists inside the editor.

Its job is automatic asset management.

This system guarantees:

- Only one database asset exists.
- The asset is always accessible.
- Missing assets are recreated automatically.

CustomMMDDataStartup

This is the automatic bootstrap system.

It runs when the editor reloads scripts or starts.

Constants

AssetName

Defines the name of the generated database asset.

Value

- Custom MMD Data.asset

Purpose

Ensures consistent naming.

ResourcesFolderName

Defines the folder where the asset should be stored.

Value

- Resources

Purpose

Allows the asset to be loaded globally at runtime.

FolderName

Defines the expected root framework folder.

Value

- MMD Collection

Purpose

Helps the system locate the framework installation directory.

Static Constructor

Runs automatically when the editor reloads assemblies.

Purpose

Schedules delayed initialization.

This avoids executing asset creation too early while Unity is still compiling.

What Happens

The system waits until the editor finishes loading, then calls the initialization routine.

Initialize Method

This is the heart of the startup system.

Its responsibility is ensuring the database exists.

Step 1 — Search Existing Assets

The system scans the entire project searching for existing database assets.

Purpose

Prevent duplicates.

If an asset already exists: Initialization immediately stops.
Step 2 — Locate Framework Folder The system searches for the "MMD Collection" folder. Purpose Determine where the database should be stored.
Step 3 — Fallback Protection If the framework folder cannot be found: The system falls back to the main Assets directory. Purpose Prevents initialization failure.
Step 4 — Ensure Resources Folder Exists Checks whether a Resources folder already exists. If missing: It creates one automatically. Purpose Guarantees the database can be loaded globally later.
Step 5 — Create Database Asset Creates the actual database object. Purpose Generate the shared configuration asset.
Step 6 — Save Asset The asset is saved into the project. Purpose Persist the database permanently.
Step 7 — Refresh Editor Forces the editor to refresh the asset database. Purpose Makes the new asset immediately visible and usable.
FindMMDCollectionFolder Method Searches the project for the framework root folder.
How It Works The system: - Searches all folders. - Compares folder names. - Returns the matching path. If no folder is found: - Returns null.
Runtime Flow Here is the complete lifecycle of the system:
Editor Opens Unity reloads assemblies. ↓ Startup System Executes Initialization is scheduled. ↓ Database Search Begins System checks for existing assets. ↓

Asset Exists?**Yes**

Nothing happens.

No

A new database asset is generated.

↓

Resources Folder Created

If necessary.

↓

Database Saved

The project now contains a valid global MMD database.

Why This Architecture Is Useful

This design provides several advantages.

Centralized Data

All MMD tools access the same information source.

No Duplicate Assets

Prevents conflicting databases.

Automatic Recovery

If the asset is deleted accidentally:

- Unity recreates it automatically.

Global Accessibility

Because the asset is stored inside Resources:

- Any runtime or editor tool can load it easily.

Localization Support

Supports both:

- Japanese pipelines.
- English workflows.

Better Tooling

The memo system improves debugging and shader organization.

Typical Usage Scenario

A common workflow would look like this:

Step 1

Import MMD model materials.

Step 2

Register materials inside the database.

Each entry receives:

- Material reference.
- Japanese name.
- English translation.
- Technical notes.

Step 3

Editor tools query the database.

For example:

- Shader converters.
- Material inspectors.
- Rendering debuggers.
- MMD importers.

Step 4

Tools automatically retrieve metadata.
No manual relinking needed.

Important Design Characteristics

Lightweight

The database only stores metadata.
No heavy rendering logic exists here.

Persistent

The asset survives:

- Scene changes.
- Play mode reloads.
- Editor restarts.

Editor Safe

Initialization only happens inside the editor.
Runtime builds do not generate assets.

Expandable

Additional fields can easily be added later.

Examples:

- Shader presets.
- Texture categories.
- Physics settings.
- Toon ramp profiles.
- Render queues.

In Summary

This script is essentially:

- A centralized MMD material database.
- A metadata container.
- A localization registry.
- An automatic asset bootstrap system.

The runtime portion stores:

- Material references.
- Names.
- Notes.
- Global flags.

The editor portion guarantees:

- Exactly one database exists.
- It is automatically created if missing.
- It is always placed in a predictable location.

The overall design is focused on:

- Stability.
- Centralization.
- Tool interoperability.
- Editor automation.
- MMD workflow organization.

Manage Objects

Overview

This system is an **editor-only visibility manager** for Unity scenes.

Its purpose is to help developers quickly organize, debug, and control groups of objects directly from the Inspector.

Instead of manually enabling or disabling many objects in the hierarchy, this component creates a centralized panel where objects can be:

- Turned on or off individually.

- Toggled all at once.
- Added through drag-and-drop.
- Removed safely with undo support.
- Hidden from the Inspector UI when not needed.

The tool is designed specifically for development workflows such as:

- Scene organization.
- Debugging systems.
- Testing environments.
- Performance comparisons.
- Showing and hiding visual helpers.
- Switching between environment setups.
- Rapid prototyping.

Outside the editor, the component destroys itself automatically so it adds no runtime cost to builds.

General Workflow

The script works in four main stages:

1. **Objects are registered** into an internal managed list.
2. The custom Inspector draws a management interface.
3. Buttons allow toggling visibility states.
4. Undo support ensures all changes are reversible.

Main Component

The main system is the ManageObjects component.

It stores:

- The list of managed objects.
- Their current states.
- Inspector display preferences.
- Utility functions for toggling and removing objects.

Serialized Class

ManageObjectsItem

This is a small data container representing one managed object.

Each entry contains:

Variable	Purpose
gameObject	Reference to the object being managed.
state	Cached visibility/active state of the object.

Serialized Variables

manageObjects

Stores all registered objects.

Each entry contains:

- A GameObject reference.
- Its current active state.

This acts as the database for the visibility manager.

Example:

Object	State
Light Rig	Enabled
Fog Volume	Disabled
Debug Grid	Enabled

GlobalState

Represents the current collective state used by the “Toggle All” feature.

When all objects are toggled:

- true means everything becomes enabled.
- false means everything becomes disabled.

This variable is hidden from the Inspector because it is controlled internally.

HideObjects

Controls whether the managed object list is visible inside the custom Inspector.

Useful when:

- The list becomes very large.
- You want a cleaner Inspector.
- You only need the global controls.

HideDefaultInspector

Controls whether Unity's default Inspector is shown.

When enabled:

- Only the custom UI is displayed.

When disabled:

- Unity also shows the raw serialized fields.

Useful for debugging or advanced editing.

Public Methods

ToggleObject

Purpose

Enables or disables a single managed object.

How it works

1. Verifies the requested index is valid.
2. Retrieves the target object entry.
3. Registers an Undo operation.
4. Inverts the cached state.
5. Applies the new state to the object.

Example

If an object is currently active:

- Calling this method disables it.

If it is disabled:

- Calling this method enables it.

ToggleAll

Purpose

Toggles all managed objects simultaneously.

How it works

1. Registers Undo operations.
2. Collects valid objects.
3. Flips the global state.
4. Applies the new state to every object.

Example

If most objects are enabled:

- Pressing the button disables all.

Pressing again:

- Re-enables all objects.

RemoveObject

Purpose

Removes an object from the management list.

Important

This does NOT destroy the object.

It only removes the reference from the visibility manager.

How it works

1. Validates the index.
2. Registers Undo support.
3. Removes the entry from the array.

Validation Method

IsValidIndex

Purpose

Prevents invalid operations.

What it checks

- The index exists.
- The entry is not null.
- The GameObject reference still exists.

This protects the tool from errors caused by deleted objects or corrupted entries.

Runtime Behavior Outside the Editor

When the project runs outside the Unity Editor:

- The component destroys itself automatically.

This ensures:

- No runtime overhead.
- No unnecessary memory usage.
- No editor-only systems inside builds.

This makes the component completely development-focused.

Custom Inspector

The second half of the script creates a fully customized Inspector interface.

This replaces Unity's normal Inspector with a specialized management panel.

Main Inspector Flow

The Inspector performs the following tasks:

1. Draws a drag-and-drop registration area.
2. Draws the global toggle button.
3. Draws configuration toggles.
4. Draws the managed object list.
5. Supports multi-object editing.

Drag & Drop Area

DrawDropArea

Purpose

Allows users to drag GameObjects directly into the Inspector.

How it works

1. Creates a large drop zone.
2. Detects drag events.
3. Accepts dropped GameObjects.
4. Adds them to all selected components.
5. Prevents duplicates.

Benefits

This makes setup extremely fast.

Instead of manually assigning objects one by one:

- You simply drag objects into the box.

Object List Drawing

DrawObjectList

Purpose

Displays every managed object with controls.

Each row contains:

Element	Purpose
Toggle button	Enables/disables the object.
Color indicator	Shows current state.
Remove button	Removes the object from the list.

Button Colors

The UI uses color feedback:

Color	Meaning
Green	Object is enabled.
Red	Object is disabled.

This makes state recognition immediate.

Multi-Object Editing

DrawBooleanMultiToggle

Purpose

Supports editing multiple selected components at once.

Unity normally struggles with mixed-value editing in custom Inspectors.

This helper:

- Detects mixed states.
- Displays Unity's mixed-value UI.
- Applies changes to all selected components.

Undo System

One of the strongest parts of this tool is the Undo integration.

Every important action records Undo operations:

Action	Undo Supported
Toggle object	Yes
Toggle all	Yes
Remove object	Yes
Add object	Yes

This makes the tool safe to use in large production scenes.

Editor UX Features

The tool includes several workflow optimizations:

Feature	Benefit
Drag-and-drop registration	Faster setup
Color-coded states	Immediate readability
Multi-object editing	Batch workflows
Undo support	Safe editing
Optional hidden lists	Cleaner Inspector
Auto-runtime removal	Zero build overhead

Typical Usage Scenarios

Debugging Systems

Enable/disable:

- Particle systems.
- AI agents.
- Lighting rigs.
- Audio emitters.
- Physics objects.

Scene Organization

Quickly hide:

- Environment layers.
- Props.
- Temporary helpers.
- Cinematic setups.

Optimization Testing

Compare performance between:

- Different lighting modes.
- Multiple environment setups.
- VFX configurations.

Tool Development

Useful for editor tool creators who need:

- Visibility presets.
- Debug object management.
- Rapid environment switching.

Example Workflow

A developer may create categories like:

Group	Objects
Lighting	Sun, Volumetrics, Reflection Probes
VFX	Smoke, Fire, Dust
Debug	Grids, Gizmos, Paths
Gameplay	NPCs, Enemies, Triggers

Then use the Inspector to instantly toggle entire groups during testing.

Overall Architecture

The system is divided into two major layers:

Layer	Responsibility
Runtime Component	Stores managed object data.
Custom Editor	Draws and controls the interface.

This separation keeps the runtime component lightweight while allowing powerful editor tooling.

Design Philosophy

The script focuses heavily on:

- Editor workflow speed.
- Safety through Undo support.
- Minimal runtime impact.
- Simple visual interaction.
- Scalable organization tools.

It behaves more like a lightweight scene management utility than a gameplay system.

Draw Mesh Instanced

Overview

This system is a lightweight **GPU instancing renderer** designed to efficiently draw multiple mesh instances in Unity.

Its main goal is to reduce rendering overhead by using **GPU Instancing**, allowing many copies of the same mesh to be rendered with fewer draw calls.

The script acts like a manual rendering manager where you define:

- Which meshes should be drawn.
- Which transforms control their placement.
- Which materials should be used.
- Whether rendering should happen constantly or only while selected in the editor.

It also includes editor visualization tools using Gizmos to help developers identify instance locations directly in the Scene view.

Main Purpose

Normally, rendering many repeated meshes individually can become expensive because each object may generate separate draw calls.

This system improves efficiency by:

- Rendering meshes directly through the graphics pipeline.
- Using instanced rendering.
- Avoiding the overhead of many separate GameObjects.
- Supporting submeshes and multiple materials.

Typical Use Cases

This system is especially useful for:

Use Case	Example
Environment decoration	Rocks, trees, props
Debug rendering	Helper meshes
Procedural tools	Runtime-generated geometry
Editor visualization	Preview objects

Large repeated assets	Crowd props, repeated structures
Lightweight rendering systems	Non-interactive scene details

Core Concept

Each entry inside the system defines:

1. A transform.
2. A mesh.
3. A material setup.

The transform determines:

- Position.
- Rotation.
- Scale.

The mesh defines what geometry is drawn.
The materials define how the mesh is rendered.

Nested Data Structure

DrawMeshInstancedList

This is the internal data container representing one instanced mesh setup.
Each entry contains everything needed to render one mesh instance group.

transform

Defines where and how the mesh appears in the world.
The transform provides:

Property	Effect
Position	Where the mesh is placed
Rotation	Which direction it faces
Scale	How large it appears

The system converts this transform into a rendering matrix before drawing.

mesh

The geometry being rendered.
This can be:

- A cube.
- Character mesh.
- Environment object.
- Any custom mesh.

The mesh may contain multiple submeshes.

materials

Defines the rendering appearance.
Each material may control:

- Shader.
- Textures.
- Transparency.
- Lighting behavior.
- Colors.

Important requirement:
The materials must support **GPU Instancing**.
Otherwise Unity cannot batch them efficiently.

Inspector Variables

OnDrawSelected

Controls when rendering occurs inside the editor.

When Enabled

Meshes are only drawn while the object is selected.
Useful for:

- Debug visualization.

- Temporary editor tools.
- Reducing Scene view clutter.

When Disabled

Meshes render continuously.

Useful for:

- Permanent rendering systems.
- Runtime visualization.
- Persistent procedural environments.

reuseMaterials

Controls how materials are assigned across submeshes.

When Enabled

Materials are reused cyclically.

Example:

Submesh	Material Used
0	Material 0
1	Material 1
2	Material 0 again
3	Material 1 again

Useful when:

- You have fewer materials than submeshes.
- You intentionally want repeated materials.

When Disabled

Rendering stops once all provided materials are consumed.

This enforces a strict one-material-per-submesh approach.

Useful when:

- Every submesh requires a unique material.
- You want stricter material control.

drawMeshInstancedLists

The main rendering database.

This list stores every instanced rendering definition.

Each entry defines:

- A transform.
- A mesh.
- A material collection.

The renderer loops through this list continuously.

Public Properties

The script exposes controlled access to important internal variables.

ReuseMaterials Property

Allows external systems or editor tools to:

- Read the reuse setting.
- Change the reuse setting dynamically.

DrawMeshInstancedLists Property

Allows external systems to:

- Add new entries.
- Remove entries.
- Replace the entire rendering list.

Useful for procedural generation systems.

Main Runtime Flow

The rendering process happens continuously through the update loop.

Each frame:

1. The renderer checks the list.
2. Validates meshes and materials.
3. Iterates through submeshes.
4. Selects materials.
5. Builds transform matrices.
6. Sends instanced draw calls to the GPU.

Update Method

Purpose

Acts as the continuous rendering trigger.
Every frame it calls the internal rendering system.

Result

The meshes remain visible and updated in real time.
This allows transforms to move dynamically.

Editor Visualization

OnDrawGizmosSelected

This method provides Scene view debugging helpers.
It only runs while the object is selected.

What It Draws

Green Sphere

A green sphere is drawn at the manager object's position.
This represents:

- The central renderer object.
- The system origin.

Red Spheres

Each managed transform receives a red sphere.
This visually indicates:

- Instance positions.
- Placement accuracy.
- Missing transforms.
- Spatial layouts.

Benefits of Gizmos

This is extremely useful for:

Benefit	Explanation
Debugging	Verify object positions
Scene organization	Understand placement visually
Procedural systems	Inspect generated layouts
Tool development	Preview instance locations

Core Rendering Logic

DrawMesh

This is the heart of the entire system.
It performs the actual GPU instanced rendering.

Rendering Flow

The method processes every registered entry.
For each entry:

Step 1 — Validate Data

The system ensures:

- A mesh exists.
- Materials exist.
- At least one material is available.

If not:

- Rendering is skipped safely.

Step 2 — Iterate Through Submeshes

Meshes may contain multiple submeshes.

Each submesh can use:

- Different materials.
- Different shaders.
- Different rendering settings.

The system processes them individually.

Step 3 — Select Material

A material is selected for the current submesh.

The selection depends on the reuse mode.

Material Reuse Logic

Reuse Enabled

The material index wraps around automatically.

This creates repeating material usage.

Reuse Disabled

Rendering stops when material count is exceeded.

This prevents accidental reuse.

Step 4 — Build Matrix Array

The transform is converted into a transformation matrix.

The matrix contains:

Data	Purpose
Position	Placement
Rotation	Orientation
Scale	Size

This matrix is sent directly to the GPU.

Step 5 — GPU Draw Call

The mesh instance is rendered using GPU instancing.

The graphics system receives:

- Mesh.
- Submesh index.
- Material.
- Transform matrix.

Then Unity submits the draw call efficiently.

GPU Instancing Advantages

Compared to normal rendering:

Traditional Rendering	GPU Instancing
Many draw calls	Fewer draw calls
More CPU overhead	Lower CPU overhead
Slower with many objects	Better scalability

Important Limitation

This script currently creates one matrix per entry.

So technically:

- It draws one instance per list item.

However, the architecture already supports expansion into:

- Multi-instance batching.
- Procedural rendering systems.
- Massive instanced arrays.

Performance Characteristics

This system is lightweight because:

- It avoids full GameObject overhead.
- Uses direct rendering calls.
- Supports GPU batching.
- Minimizes scene hierarchy complexity.

Best Usage Scenarios Procedural Rendering

Excellent for:

- Generated forests.
- Scattered props.
- Runtime decoration.

Debug Systems

Useful for:

- Navigation markers.
- Debug geometry.
- Collision visualization.

Editor Tools

Great for:

- Preview systems.
- Scene helper objects.
- Custom visualization tools.

Architectural Design

The script is divided into:

Layer	Responsibility
Data Layer	Stores mesh instance definitions
Rendering Layer	Sends GPU draw calls
Editor Visualization Layer	Draws gizmos

Design Philosophy

The system focuses on:

- Simplicity.
- Lightweight rendering.
- Editor usability.
- GPU efficiency.
- Flexible material assignment.

It behaves more like a low-level rendering utility than a gameplay system.

Custom MMD Data Utility

Overview

This utility acts as the **central access layer** for the MMD Collection database system.

Its purpose is to provide:

- Safe loading of the global MMD database.
- Cached access for better performance.
- Cleanup and validation helpers.
- A single consistent entry point for all systems that need MMD material metadata.

Instead of every tool loading the database independently, this utility centralizes the process.

This improves:

- Performance.
- Maintainability.
- Stability.
- Consistency across the framework.

Main Responsibility

The utility performs two primary jobs:

Responsibility	Description
Database Access	Loads and returns the global database asset.
Database Maintenance	Removes invalid or broken material entries.

General Workflow
The system follows this flow:

1. A tool requests access to the MMD database.
2. The utility checks whether the database is already cached.
3. If cached, it immediately returns the stored reference.
4. If not cached, it loads the asset from the Resources folder.
5. The loaded asset is cached for future use.
6. Validation helpers can later clean invalid entries.

Core Concept
This class exists because repeatedly loading assets from the Resources system can become inefficient. Without caching:

- Every request would trigger another lookup.
- Performance would decrease.
- Memory operations would increase unnecessarily.

This utility solves that problem by storing the loaded asset in memory after the first access.

Static Utility Architecture
The utility is static, meaning:

- It does not need to exist as a scene object.
- No component attachment is required.
- It behaves like a globally accessible service.

This makes it ideal for framework-wide database access.

Cached Variable
cachedData
Purpose
Stores the already-loaded database asset in memory.

Why It Exists
Loading from the Resources system repeatedly is slower than reusing an already-loaded reference. The cache avoids:

- Repeated disk lookups.
- Repeated memory allocations.
- Unnecessary asset loading operations.

Lifecycle
First Access
The cache is empty.
The utility loads the asset and stores it.

Future Accesses
The cached version is returned instantly.
No additional loading occurs.

Main Public Method
GetCustomMMDData
Purpose
Retrieves the global MMD database asset safely.
This is the main entry point used by other systems.

Full Workflow
Step 1 — Check Cache
The utility first checks whether the database has already been loaded.
If it exists:

- The cached version is returned immediately.
This is the fast path.

Step 2 — Load From Resources

If the cache is empty:

- The utility searches the Resources folder.

It specifically looks for:

Custom MMD Data

This must match the asset name exactly.

Why Resources Is Used

Using Resources provides:

Benefit	Explanation
Global accessibility	Any system can load the asset easily
Simplicity	No manual references required
Centralization	One shared database
Predictable loading	Standardized access path

Step 3 — Validation

After attempting to load:

- The utility verifies the asset actually exists.

If loading fails:

- An error message is logged.

This helps developers identify:

- Missing assets.
- Incorrect folder placement.
- Naming problems.

Step 4 — Cache Storage

If loading succeeds:

- The asset is stored in cachedData.

This allows future requests to bypass loading completely.

Step 5 — Return Database

The database reference is returned to the caller.

Other systems can then:

- Read material metadata.
- Access configuration flags.
- Query localized material names.
- Use shader configuration data.

Usage Examples

This utility would commonly be used by systems such as:

System	Purpose
Material editors	Access metadata
Shader tools	Read configuration
Rendering systems	Query MMD material data
Localization tools	Access JP/EN names
Validation systems	Check database integrity

Maintenance Method

RemoveInvalidMaterials

Purpose

Cleans the database by removing broken material entries.

This helps maintain long-term database integrity.

Why Cleanup Is Necessary

Over time:

- Materials may be deleted.
- Assets may become missing.
- References may break.
- Imported assets may change.

This can leave invalid entries inside the database.

What Counts as Invalid

An entry becomes invalid when:

- Its material reference no longer exists.

Example:

Entry	Result
Valid material assigned	Kept
Missing/deleted material	Removed

Cleanup Workflow

Step 1 — Validate Database

The utility first checks whether the database itself exists.

If not:

- Cleanup stops safely.

Step 2 — Scan Material List

The utility iterates through all stored material entries.

Step 3 — Remove Invalid Entries

Any entry with a missing material reference is deleted from the list.

This prevents:

- Null reference errors.
- Corrupted metadata.
- Broken editor tools.

Benefits of Automatic Cleanup

Benefit	Explanation
Prevents invalid references	Avoids runtime/editor errors
Keeps database clean	Removes obsolete entries
Improves tool reliability	Ensures valid data
Reduces debugging time	Eliminates hidden corruption

Relationship With Other Systems

This utility sits between:

Layer	Responsibility
Database Asset	Stores raw metadata
Utility Layer	Provides access and maintenance
Framework Tools	Consume the data

This creates a clean architecture.

Design Philosophy

The utility focuses heavily on:

- Centralized access.
- Performance through caching.
- Database integrity.
- Simplicity.
- Framework-wide consistency.

It behaves similarly to a lightweight database manager for the entire MMD Collection framework.

Why This Architecture Is Good

Without this utility:

- Every system would load assets independently.
- Cleanup logic would be duplicated.

- Error handling would be inconsistent.

With this architecture:

- All systems share the same database instance.
- Loading logic exists in one place.
- Validation remains centralized.
- Maintenance becomes easier.

Overall System Role

This class is essentially the **gateway** to the entire MMD material metadata system.

Almost every framework tool that needs MMD database access would rely on this utility to:

- Retrieve the database safely.
- Avoid repeated loading costs.
- Keep stored metadata clean and reliable.

MMD Bone Renderer

Overview

This system is an editor visualization tool designed to display bone hierarchies directly inside the Unity Scene view.

Its main purpose is to help developers inspect skeletal structures visually during animation setup, rig debugging, MMD model validation, or general scene inspection.

Instead of relying only on the Hierarchy window, the component draws:

- Bone connections as lines.
- Bone joints as visual points.
- Bone groups using different colors.
- Bone tips (end bones without children).

The system works entirely in editor mode and is intended for debugging and visualization purposes rather than gameplay logic.

Main Purpose of the System

The script acts like a skeletal visualization layer.

It analyzes a collection of transforms that represent bones and automatically determines:

- Which bones are connected.
- Which transforms are terminal tips.
- Which bones belong to specific visual groups.

The extracted information can then be used by custom editor tools or Scene rendering systems to draw the skeleton.

Main Data Structure

BoneRendererData

This is a container representing a group of bones.

Each group contains:

- A visualization color.
- A list of bones assigned to that group.

This allows multiple skeleton regions to be separated visually.

Example:

- Blue group → body bones.
- Orange group → facial bones.
- Green group → accessories.

Variables

drawBones

Determines whether bone rendering is enabled.

If disabled:

- The visualization system stops drawing the skeleton.
- Bone extraction still exists internally, but rendering systems can ignore it.

Typical usage:

- Quickly enabling/disabling visual clutter.
- Temporarily hiding skeleton overlays.

jointSize

Controls the visual size of the bone joints in the Scene view.

Higher values:

- Larger joint markers.
- Easier visibility at distance.

Lower values:

- More precise visualization.
- Less screen clutter.

boneRendererList

Main collection of all bone groups.

Each group contains:

- A color.
- A list of bones.

This is the core data source used during extraction.

m_Bones

Internal cache containing all valid parent-child bone connections.

Each entry stores:

- Parent transform.
- Child transform.

These pairs are later used to draw bone lines.

Example:

UpperArm → Forearm

Forearm → Hand

Hand → Finger

m_Tips

Internal cache containing bones without valid children.

These are considered "tips" or terminal points.

Examples:

- Fingertips.
- Hair ends.
- Tail ends.
- Accessory ends.

Public Properties

The properties expose internal settings safely to other systems.

DrawBones

Allows external systems to enable or disable visualization.

Useful for:

- Custom editor windows.
- Debug toggles.
- Scene tools.

JointSize

Allows runtime or editor adjustment of joint size.

BoneRendererList

Provides access to the complete bone group collection.

Other systems can:

- Add groups.
- Remove groups.
- Modify colors.
- Replace bones.

Bones

Returns the extracted bone connection pairs.
This is usually consumed by rendering systems that draw lines.

Tips

Returns the extracted tip transforms.

Useful for:

- Drawing end markers.
- Detecting isolated chains.
- Debugging incomplete rigs.

Events

The system includes global callbacks.

onAddBoneRenderer

Triggered whenever a renderer becomes active.

Useful for editor tools that maintain a global registry of active bone renderers.

onRemoveBoneRenderer

Triggered whenever a renderer is disabled.

Useful for cleanup systems.

Lifecycle Methods

OnEnable

Executed when the component becomes active.

It performs two actions:

1. Rebuilds all bone data.
2. Notifies listeners that a renderer was added.

This guarantees the renderer always starts with valid cached data.

OnDisable

Executed when the component becomes inactive.

It only notifies listeners that the renderer was removed.

Public API Methods

Invalidate

Forces a complete rebuild of the cached bone data.

Used whenever:

- Bones change.
- Groups change.
- Hierarchies change.
- Visibility changes.

Think of it as:

"Refresh the skeleton data."

Reset

Clears all cached bone information.

After calling this:

- No bone connections remain.
- No tip data remains.

Useful for:

- Cleanup.
- Reinitialization.
- Tool resets.

AddBoneGroup

Creates an entirely new bone group.

Parameters:

- Visualization color.
- List of bones.

The method:

1. Creates the group.
2. Ensures lists exist safely.
3. Adds the group.
4. Rebuilds the internal caches.

Typical use:

Create a facial bone group in red.

Create a body bone group in blue.

AddBone

Adds a single bone into an existing group.

The method safely validates:

- Group existence.
- Bone validity.
- Duplicate prevention.

After insertion:

- Bone relationships are recalculated.

This allows incremental construction of skeleton groups.

Internal Bone Extraction System

ClearBones

Removes all cached extracted data.

This resets:

- Bone pairs.
- Bone tips.

ExtractBones

This is the heart of the system.

It analyzes all registered bones and automatically determines hierarchy relationships.

The process works in several stages.

Stage 1 — Validation

The method first checks whether groups exist.

If not:

- Everything is cleared.
- Processing stops.

Stage 2 — Build Fast Lookup Table

All valid transforms are collected into a hash set.

Purpose:

- Extremely fast child validation.
- Avoid expensive repeated searches.

This allows the system to quickly answer:

"Is this child transform also a registered bone?"

Stage 3 — Iterate Through All Bones

Every registered bone is analyzed.

Invalid entries are skipped automatically.

Stage 4 — Visibility Filtering

The system ignores:

- Hidden objects.
- Invisible layers.

This prevents unnecessary rendering of hidden skeleton parts.

Stage 5 — Child Relationship Detection

For every transform:

- All children are inspected.
- Valid registered children become bone connections.

Each valid connection becomes a TransformPair.

Example:

Spine → Chest

Chest → Neck

Neck → Head

Stage 6 — Tip Detection

If a bone has no valid child connections:

- It becomes a tip.

This is important because end bones often require different rendering visuals.

Stage 7 — Cache Final Data

The generated lists are converted into arrays and cached.

Result:

- Fast rendering access.
- Reduced allocations.
- Efficient editor drawing.

AltoGetBones

This is an automatic setup utility.

It scans all child transforms and converts them into a bone group automatically.

The method is intentionally marked as:

"Not the recommended workflow."

Because it blindly collects everything.

What It Does

1. Displays a confirmation dialog.
2. Scans all children.
3. Creates bone entries automatically.
4. Replaces all existing groups.
5. Rebuilds extraction data.

Typical Usage

Useful for:

- Fast debugging.
- Initial setup.
- Unknown rigs.
- Temporary visualization.

Not ideal for production organization.

Usage Workflow

Typical Setup

Step 1

Add the component to a root skeleton object.

Step 2

Create bone groups.

Example:

Group 1 → Main body.

Group 2 → Face.

Group 3 → Hair.

Step 3

Assign transforms into groups.

Step 4

Enable visualization.

The Scene view will now display:

- Bone lines.

- Joint points.
- Bone tips.

Practical Use Cases

Rig Debugging

Visualize broken bone hierarchies quickly.

MMD Validation

Check imported MMD skeleton integrity.

Animation Inspection

Inspect joint flow during animation playback.

Tool Development

Acts as a data provider for custom editor renderers.

Technical Artists

Useful for validating:

- Bone parenting.
- Hidden transforms.
- Layer visibility.
- End bones.

Important Design Characteristics

Editor-Only Focus

The system is designed primarily for editor visualization.

It is not intended as a runtime gameplay system.

Cached Architecture

Bone extraction is cached to avoid expensive recalculation every frame.

Layer Awareness

The system respects:

- Scene visibility.
- Layer visibility.

This integrates properly with Unity editor workflows.

Modular Group System

Multiple groups allow:

- Visual organization.
- Colored regions.
- Partial skeleton debugging.

Automatic Hierarchy Detection

You do not manually define lines between bones.

The system derives relationships directly from transform parenting.

Final Summary

This component is essentially a skeletal visualization and hierarchy analysis tool for Unity editor workflows.

Its responsibilities include:

- Organizing bones into groups.
- Detecting parent-child relationships.
- Detecting terminal bones.
- Caching skeletal structure data.
- Providing visualization-ready information.
- Supporting editor debugging and MMD workflows.

The system is optimized for:

- Clarity.

- Fast iteration.
- Rig inspection.
- Debugging large skeletal structures.

Bones Transform

Overview

This system defines a reusable bone configuration container together with a custom Inspector interface for Unity editor tools.

The script has two main responsibilities:

1. Store information about a single bone and its processing settings.
2. Render a clean and organized custom Inspector UI for editing those settings.

It is designed to be used by larger systems such as:

- Bone visualizers.
- Animation tools.
- MMD rig systems.
- Synchronization systems.
- Pose editors.
- Retargeting tools.

The script itself does not move or animate bones directly.

Instead, it acts as a configuration layer that other systems can read and interpret.

Main Concept

Each entry represents one bone together with three processing flags:

- Rotation enabled.
- Movement enabled.
- Visibility enabled.

This allows external systems to decide what operations should affect each bone.

Example:

Head Bone:

- Rotate = true
- Move = false
- Visible = true

This could mean:

- Rotation is allowed.
- Position movement is ignored.
- The bone should appear in visualization tools.

Main Data Class

BonesTransform

This is the core data container.

It stores:

- A reference to a transform.
- Processing flags.
- Visualization flags.

Because the class is serializable, Unity can save it inside:

- Components.
- Assets.
- Lists.
- Arrays.
- ScriptableObjects.

Variables

bone

Reference to the target transform representing a skeletal bone.

This is the actual object that external systems will process.

Examples:

- Head.
- Arm.

- Spine.
- Finger.
- Hair bone.

Without this reference, the configuration entry has no valid target.

rotate

Determines whether rotation operations are allowed for this bone.

If enabled:

- External systems may rotate the bone.
- Animation systems may synchronize rotations.
- Visualization systems may include rotation data.

If disabled:

- Rotation should be ignored.

Typical Usage

Useful for selective animation control.

Example:

Finger bones rotate.

Accessory bones do not.

move

Determines whether positional movement is allowed for the bone.

If enabled:

- Position changes may be applied.
- Translation synchronization is allowed.

If disabled:

- The bone should remain positionally static.

Typical Usage

Very useful in animation retargeting.

Example:

Spine can move.

Eyes only rotate.

visible

Determines whether the bone should be visually rendered or processed by visualization systems.

If enabled:

- Bone renderers may draw it.
- Debug tools may display it.

If disabled:

- The bone becomes hidden from visualization systems.

This does not necessarily disable the transform itself.

It only affects systems that respect this flag.

Custom Inspector Drawer

BonesTransformDrawer

This is the editor UI system responsible for drawing the bone configuration cleanly inside the Inspector.

Without this drawer:

- Unity would draw fields vertically using the default layout.
- The UI would be less compact and harder to read.

The custom drawer creates a structured layout.

Purpose of the Drawer

The drawer improves:

- Readability.
- Compactness.
- Workflow speed.
- Bone editing efficiency.

Instead of large stacked fields, it creates:

Bone: [Transform]

[Rotate] [Move] [Visible]

This makes editing many bones significantly faster.

Main Editor Methods

OnGUI

This is the rendering method for the custom Inspector interface.

It is responsible for drawing the entire visual layout.

What the Method Does

The method performs several stages.

Stage 1 — Begin Property Drawing

The system initializes the property drawing context.

This ensures:

- Proper prefab override support.
- Undo support.
- Serialized property handling.

Stage 2 — Calculate Layout Values

The method calculates:

- Line height.
- Vertical spacing.
- Toggle widths.

These values ensure consistent alignment with Unity's editor style.

Stage 3 — Draw Label

A top label is drawn for the entire property block.

Example:

Bone 0

or

Spine Bone

depending on how the property is used.

Stage 4 — Draw Bone Field

The transform reference selector is rendered.

This allows the user to:

- Drag a transform.
- Select from scene hierarchy.
- Assign bones.

This is the main target assignment field.

Stage 5 — Draw Toggle Row

The three settings are rendered side-by-side:

- Rotate.
- Move.
- Visible.

The layout is compact and evenly divided.

This creates a cleaner workflow when editing large bone collections.

Stage 6 — Temporary Label Width Adjustment

The drawer temporarily reduces label width.

Purpose:

- Keep toggle rows compact.
- Prevent oversized labels.
- Improve alignment.

After drawing:

- Original label width is restored safely.

Stage 7 — Finish Property Drawing

The property context is closed properly.

This finalizes:

- Serialization.
- Undo integration.
- Prefab override tracking.

GetPropertyHeight

This method tells Unity how much vertical space the property requires.

Because the drawer uses multiple rows, Unity needs an exact height calculation.

Why This Is Necessary

Without proper height reporting:

- Controls would overlap.
- Layout corruption could occur.
- Inspector spacing would break.

Height Structure

The layout uses:

- One line for the label.
- One line for the bone field.
- One line for the toggles.
- Spacing between rows.

Practical Usage

Example Use Case — Bone Renderer

A visualization system may read:

Visible = true

to decide whether a bone should be drawn.

Example Use Case — Animation Synchronization

A retargeting system may read:

Rotate = true

Move = false

meaning:

- Apply rotations.
- Ignore positional movement.

Example Use Case — Physics Systems

Certain bones may allow movement while others remain locked.

Example:

Hair tips move.

Spine remains fixed.

Example Use Case — Partial Rig Control

You can selectively enable operations per bone.

Example:

Face:

- Rotate = true
- Move = false

Root:

- Rotate = false
- Move = true

Design Characteristics

Lightweight

The class contains only minimal configuration data.

This keeps it highly reusable.

Reusable

The structure can be embedded into:

- Lists.
- Arrays.
- ScriptableObjects.
- Runtime components.

Editor-Friendly

The custom drawer greatly improves usability for large skeletons.

Modular

The system does not enforce behavior.

It only describes configuration.

Other systems decide how to interpret the flags.

Scalable

Works efficiently even with:

- Hundreds of bones.
- Multiple groups.
- Large MMD rigs.

Relationship With Other Systems

This script is commonly used together with:

- Bone visualization systems.
- Rig editing tools.
- MMD skeletal systems.
- Animation retargeting tools.
- Pose synchronization systems.

It acts as a foundational configuration building block.

Final Summary

This script provides:

- A serializable bone configuration structure.
- Bone transform references.
- Rotation/movement/visibility control flags.
- A highly optimized custom Inspector interface.

The main goal is to simplify the management of large skeletal structures while giving external systems precise control over how each bone should behave.

It is essentially:

A configurable bone descriptor with an optimized editor UI.

MMD Bone Renderer Inspector

Overview

This script is a custom Inspector extension for a bone visualization system.

Its purpose is to ensure that whenever the user modifies the MMDBoneRenderer component in the Unity Inspector, the internal bone data is automatically recalculated and kept in sync.

In simpler terms:

- It watches the Inspector for changes.
- It detects Undo/Redo actions.
- It refreshes the bone hierarchy data automatically when needed.

This prevents stale or outdated bone visualization data.

Main Purpose

Normally, Unity only updates serialized fields when the user changes values.

However, this system needs extra behavior:

- When bone lists change → rebuild bone connections.

- When groups change → update cached arrays.
 - When Undo/Redo happens → restore correct internal state.
- So this Inspector ensures the runtime/editor data stays consistent.

Class Role

This class replaces the default Inspector UI for the MMDBoneRenderer component. It does not draw custom UI elements.

Instead, it:

- Uses the default Inspector layout.
- Adds automatic update logic behind the scenes.

So the user experience remains familiar while gaining extra functionality.

Core Workflow

Every time the Inspector is drawn, the script performs this cycle:

1. Render all default fields.
2. Detect if anything changed.
3. Detect Undo/Redo events.
4. If changes occurred → rebuild bone data.

Variables and Internal Concepts

This script does not define persistent variables.

Instead, it relies on temporary Inspector state and Unity's built-in systems.

However, it uses important internal concepts:

targets

This represents all selected objects in the Inspector.

Usage:

- Supports multi-object editing.
- Allows batch updates.
- Ensures consistency across selections.

Example:

If 3 MMDBoneRenderer objects are selected:

- All 3 will be updated simultaneously when changes occur.

boneRendererDirty

This is a temporary flag used to determine whether the system needs to refresh data.

It becomes true when:

- The user changes a field in the Inspector.
- An Undo or Redo action is performed.

It acts as a trigger for recalculation.

Event.current

This represents the current Unity editor event being processed.

It is used to detect special actions like:

- Undo.
- Redo.
- Validation commands.

Methods

OnInspectorGUI

This is the main method responsible for:

- Drawing the Inspector UI.
- Monitoring changes.
- Triggering bone extraction updates.

It runs every time Unity redraws the Inspector window.

Step-by-step Behavior

Step 1 — Start Change Detection

The system begins monitoring modifications using a change tracking mechanism.

This allows Unity to detect:

- Field edits.
- List modifications.
- Serialized property updates.

Step 2 — Draw Default Inspector

Instead of creating a custom UI, it renders Unity's built-in Inspector layout.

This includes:

- Bone lists.
- Settings fields.
- Configuration options.

So the UI remains identical to the standard component view.

Step 3 — End Change Detection

After drawing, the system checks whether anything changed.

If the user modified anything:

- `boneRendererDirty` becomes true.

Step 4 — Detect Undo/Redo

Even if no direct Inspector change occurred, the system also checks:

- Whether an Undo or Redo command was executed.

This is important because:

- Undo/Redo changes data without triggering normal Inspector change detection.

So this ensures consistency after editing history changes.

Step 5 — Trigger Bone Recalculation

If any change is detected, the script loops through all selected objects.

For each `MMDBoneRenderer`:

- It calls the bone extraction method.

This rebuilds:

- Bone connections.
- Tip lists.
- Internal cached arrays.

Key Method Call

ExtractBones()

This is the core refresh operation from the `MMDBoneRenderer` system.

It:

- Recalculates parent-child bone relationships.
- Updates cached bone arrays.
- Rebuilds tip detection.
- Ensures visualization matches current data.

In this Inspector context, it is used as an automatic synchronization step.

Usage Scenario

Example 1 — Editing Bone Groups

User changes:

- Adds a new bone.
- Removes a bone.
- Changes group settings.

Result:

- Inspector detects change.
- `ExtractBones` is called.
- Scene visualization updates instantly.

Example 2 — Undo Action

User presses Undo after removing a bone.

Result:

- Unity restores previous state.

- This script detects UndoRedoPerformed.
- Bone data is rebuilt automatically.

Example 3 — Multi-object Editing

User selects multiple MMDBoneRenderer components.

Any change triggers:

- All selected renderers are updated.
- Each one recalculates independently.

Design Characteristics

Lightweight

No persistent data is stored.

The script only reacts to events.

Non-intrusive

It does not replace the Inspector UI.

It only extends behavior behind the scenes.

Event-driven

Updates only happen when necessary:

- User edits.
- Undo/Redo occurs.

This avoids unnecessary recalculations.

Multi-object safe

Uses Unity's target system to ensure:

- Multiple components stay synchronized.

Why This Script Exists

Without this system:

- Bone visualization would become outdated after edits.
- Undo/Redo would break consistency.
- Users would need to manually refresh data.

With this script:

- Everything stays automatically synchronized.
- No manual refresh required.
- Editing workflow becomes smoother and safer.

Final Summary

This custom Inspector acts as an automatic synchronization layer for the MMDBoneRenderer system.

Its responsibilities are:

- Draw the default Inspector UI.
- Detect user modifications.
- Detect Undo/Redo operations.
- Automatically refresh bone hierarchy data.

In short:

It ensures that any change in the Inspector immediately updates the bone visualization system.

MMD Bone Renderer Utils

This script is a **full editor-only bone visualization and interaction system** for Unity. It builds a GPU-instanced skeleton view in the Scene view, lets you select bones like real editor handles, and manages multiple bone-rendering components at once.

Below is a structured explanation of how it works, focusing on **what each variable does and how each method is used**, without using C# syntax.

High-Level Idea

The system:

- Registers all bone renderer components in the scene.

- Extracts bone relationships (parent → child).
- Builds a GPU instanced pyramid mesh for each bone.
- Draws bones efficiently in the Scene view.
- Adds interaction (click, hover, drag, selection).
- Syncs with Unity selection and transform tools.

Nested Type: BatchRenderer

This is the **core GPU instancing manager**.

◆ **mesh**

The base mesh used for every bone instance.

- In this case: a **pyramid shape**
- Represents a bone visually (thicker at base, pointed tip)

◆ **material**

Material used to render all bone instances.

- Supports color and highlight changes
- Has instancing enabled

◆ **kMaxDrawMeshInstanceCount**

Maximum number of instances per GPU draw call.

- Value: **1023**
- Unity limitation for instanced rendering batches

◆ **m_Matrices**

Stores transformation data for each bone instance.

Each entry defines:

- position
- rotation
- scale

Used to place bones in world space.

◆ **m_Colors**

Stores base color for each bone instance.

Used for:

- group coloring
- visual distinction between bones

◆ **m_Highlights**

Stores highlight color for interaction state.

Used for:

- hover effect
- selection effect

AddInstance

Adds a new bone to be rendered.

Takes:

- transformation
- base color
- highlight color

Stores them into internal lists so they can be drawn later in bulk.

Clear

Removes all queued bone instances.

Used every frame before rebuilding the skeleton.

RenderChunkCount

Calculates how many GPU draw batches are needed.

Why:

- Unity limits instancing per draw call

- So data must be split into chunks

⚙ **GetRenderChunk**

Extracts a slice of data for a specific chunk.

Used to:

- split large bone arrays
- feed Unity manageable batches

🌀 **Render**

This is where GPU drawing happens.

Steps:

1. Checks if there is anything to render
2. Activates material
3. Splits data into chunks
4. For each chunk:
 - draws solid bone mesh
 - draws wireframe overlay
5. Sends commands to GPU using command buffer

Result:

- very fast rendering even with many bones

📌 **Static State Section**

This controls global system behavior.

◆ **s_BoneRendererComponents**

A list of all active bone renderer components in the scene.

- Every MMDBoneRenderer registers here
- Used as the master scene database

◆ **s_PyramidMeshRenderer**

The shared GPU renderer for bone meshes.

- Lazily created (only when needed)
- Reused across all bone systems

◆ **s_Material**

Shared material used for rendering bones.

- Loaded once
- Stored globally
- Prevents duplication and improves performance

◆ **s_VisibleLayersCache**

Stores current visible layers from Unity editor.

- Used to detect changes in layer visibility
- Triggers refresh when modified

◆ **s_ButtonHash**

Unique ID used for Scene GUI interaction.

- Ensures correct selection tracking per bone

🔧 **Constants**

◆ **k_Epsilon**

Very small threshold value.

- Used to detect “zero-length bones”
- Identifies bone tips

◆ **k_BoneBaseSize**

Controls thickness of bones at base.

- Scales pyramid width

◆ **k_BoneTipSize**

Controls size of bone tip visualization.

- Used for interaction radius

⚙ **Static Constructor**

Runs automatically when Unity loads the editor.

It:

- Registers bone renderer add/remove events
- Hooks into Scene view rendering loop
- Hooks into hierarchy and visibility changes
- Stores initial visible layer state

👉 This makes the system fully automatic.

🧠 **Lazy Resources**

◆ **Material getter**

Creates or returns shared material.

If missing:

- loads shader named “BoneHandles.shader”
- creates material
- enables instancing

◆ **PyramidMeshRenderer getter**

Creates the bone mesh system.

It builds:

Geometry:

- pyramid vertices (tip + base)

Submeshes:

- solid faces
- wireframe lines

Then attaches it to BatchRenderer.

👉 This defines how every bone looks.

🎨 **ComputeBoneMatrix**

This is the math engine of bone rendering.

It:

1. Calculates direction from start to end
2. Builds a perpendicular coordinate system
3. Computes scale based on bone length
4. Moves pivot to center of bone

Result:

A transformation matrix that turns a simple pyramid into a correctly oriented bone.

🖼 **DrawSkeletons (Main Scene Renderer)**

This runs every Scene view frame.

It:

1. Checks if visibility layers changed
2. Clears previous frame bones
3. Loops through all bone renderer components
4. Filters invalid or hidden objects
5. Builds visible bone groups
6. Renders:
 - bone connections
 - bone tips
 - interactive handles
7. Sends everything to GPU renderer

Bone rendering flow

For each component:

1. Group filtering

Only bones marked visible are processed.

2. Bone lines

Draw connections between parent and child transforms.

3. Tips

Draw endpoints that have no children.

4. Handles

Draw interactive gizmos:

- cube if move enabled
- sphere otherwise

AreBonesInSameGroup

Checks if two bones belong to the same group.

Used to:

- avoid drawing invalid connections
- enforce grouping rules

It scans all groups and checks membership.

Registration Methods

OnAddBoneRenderer

Adds a component to global list.

OnRemoveBoneRenderer

Removes it from list.

👉 This keeps scene tracking updated automatically.

Refresh Methods

OnVisibilityChanged

Called when:

- layer visibility changes
- scene visibility updates

It:

- forces all bones to rebuild
- refreshes Scene view

OnHierarchyChanged

Called when:

- objects are added/removed
- hierarchy changes

Same behavior: rebuild everything

DoBoneRender (Core Interaction + Rendering)

This is the most complex function.

It handles a **single bone instance**.

Step 1: Define bone endpoints

- start = bone position
- end = child position or same position

Step 2: Identify bone type

If length is near zero:

- it's a tip bone

Step 3: Create Scene interaction ID

Each bone gets a unique control ID so Unity can:

- detect hover
- detect clicks
- manage drag

Step 4: Event handling system

It reacts to:

Mouse hover

- highlights bone

Mouse click

- selects bone
- switches Unity tool:
 - move
 - rotate
 - transform

Mouse drag

- starts drag-and-drop operation

Mouse release

- releases selection control

Step 5: Rendering (Repaint)

Decides final color:

- normal color
- hover highlight
- selection highlight

Then:

- ✓ sends bone instance to GPU renderer
- ✓ uses computed matrix
- ✓ queues rendering

Final Behavior Summary

When Unity Scene view runs:

1. Bones are registered automatically
2. Transform hierarchy is analyzed
3. Bone connections are built
4. GPU instancing renders all bones efficiently
5. Handles allow selection and editing
6. Unity tools automatically adapt to bone settings

In Simple Terms

This system is basically:

A real-time GPU-powered skeleton editor that behaves like Unity's built-in transform tools, but specialized for MMD-style bone hierarchies.

Shader Keyword Checker

This script is a **Unity Editor utility tool** that inspects selected materials and prints their **global shader keywords**. It uses reflection to access an internal Unity API that is not normally exposed.

Overall Purpose

The script adds a **context menu button** inside Unity's **Project window**:

"Check Shader Keywords"

When triggered, it:

- looks at selected assets
- filters only materials
- reads their shader
- extracts global shader keywords
- prints them in the console

Namespace / Scope

This tool lives inside an **editor-only utility namespace**, meaning:

- it only runs inside Unity Editor

- it is not included in builds
- it is used for debugging shaders and rendering behavior

Core Variable

◆ **getShaderGlobalKeywordsMethod**

This is a cached reference to an internal Unity function.

What it represents:

A hidden Unity editor function that can retrieve:

- all global shader keywords for a shader
- keywords affecting rendering variants and feature toggles

Why caching is important:

Without caching:

- reflection would run every time
- performance would be worse
- code would repeatedly search Unity internals

With caching:

- method is found once
- reused for all shader checks

Menu Method

◆ **CheckShaderKeywords**

This is the **entry point of the tool**.

It is triggered from:

Unity → Assets → Tools → MMD Collection → Check Shader Keywords

What it does step-by-step:

1. Iterates selected objects

It scans everything currently selected in the Project window.

This can include:

- materials
- textures
- models
- folders

2. Filters only materials

For each selected item:

- if it is not a material → warning is printed
- only materials continue processing

3. Gets shader from material

Each material is checked for its shader:

- if missing → error is logged
- if present → continues

4. Calls keyword logging system

Each valid shader is sent to the utility function that reads keywords.

Utility Method

◆ **LogGlobalShaderKeywords**

This is the core inspection function.

It takes a shader and prints its global keywords.

Step 1: Safety check

If shader is null:

- function exits immediately

Prevents crashes from invalid inputs.

Step 2: Lazy initialization of reflection

If the cached method is not yet set:

It searches inside Unity internals:

- looks for a hidden function called:
“GetShaderGlobalKeywords”

It uses reflection because:

- Unity does NOT expose this API publicly
- it exists only inside editor internals

If it fails:

- error is logged
- execution stops

Step 3: Invoke internal Unity function

Once found, it calls the method:

- passes the shader as input
- receives a result containing keyword data

Step 4: Validate result

The result is checked:

Case A: no keywords

- prints:
“Shader has no global keywords”

Case B: keywords exist

- converts list into readable text
- prints each keyword on a new line

What are “global shader keywords”?

They are **global feature switches in shaders**, used for things like:

- enabling lighting models
- toggling effects (fog, shadows, etc.)
- activating rendering variants
- controlling shader compilation paths

They influence how Unity compiles and selects shader variants.

How to use this tool

Step-by-step usage:

1. Open Unity Project window
2. Select one or more materials
3. Right-click or open Assets menu
4. Click:
“Tools → MMD Collection → Check Shader Keywords”
5. Check Console output

Output behavior

For each material:

If valid:

- shader name is shown
- list of global keywords is printed

If invalid:

- warnings or errors appear in console

Why this script is useful

It helps developers:

- debug shader variants
- understand rendering behavior
- check feature flags in materials
- verify shader configuration problems

Key design idea

This script is essentially:
A reflection-based shader debugging inspector for Unity materials.
It extends Unity's editor without modifying Unity itself.

Create Prefab From Model

This script is an **Editor utility for Unity** that automates converting selected 3D model assets into prefabs by extracting their mesh components and rebuilding them as standalone GameObjects.

Overall Purpose

When you select one or more 3D models in the Unity Project window and run the menu command, the script:

- Reads the selected models.
- Finds all mesh-related components inside them.
- Creates new GameObjects that replicate those meshes and materials.
- Combines those objects into a prefab.
- Saves the prefab next to the original model.

It is purely **editor-side automation**, meaning it is not used during gameplay.

Core Workflow (High Level)

1. User selects model assets.
2. Menu command runs.
3. Script filters valid model prefabs.
4. Each model is processed:
 - Extract meshes from SkinnedMeshRenderer and MeshFilter
 - Create new GameObjects
 - Assign mesh + materials
5. Combine everything into a prefab.
6. Save prefab into the same folder as the model.

Methods Explained

1. Main Entry Point (Menu Command)

Purpose

This is the function triggered from Unity's menu:

- "Assets → Tools → MMD Collection → Create Prefabs From Selected Model"

What it does

- Reads everything currently selected in the Project window.
- Filters only valid **3D model assets**.
- Converts each valid model into a prefab.

Key logic

- If nothing valid is selected → logs a warning and stops.
- If valid models exist → sends each one to the conversion function.

Usage

You use it by selecting FBX / 3D model files and clicking the menu option.

2. Convert Model to Prefab

Purpose

Takes a single model and prepares it for prefab conversion.

What it does

- Searches inside the model for:
 - SkinnedMeshRenderer components (animated meshes)
 - MeshFilter components (static meshes)
- Converts both types into GameObjects.
- Collects all created objects.
- Sends them to prefab creation.

Output

A list of new GameObjects representing the model's geometry.

3. Generic Component Conversion Function

Purpose

This is the extraction engine.

It processes:

- Skinned mesh objects (characters, animated rigs)
- Static mesh objects (props, environment)

What it does step-by-step

For each component:

1. Checks if it is valid (not null).
2. Determines type:
 - If it is a skinned mesh:
 - Takes mesh data from the skinned renderer
 - Takes its materials
 - If it is a static mesh:
 - Gets mesh from MeshFilter
 - Gets materials from MeshRenderer
3. If valid data exists:
 - Creates a new GameObject
 - Assigns mesh and materials
 - Adds it to the output list

Important behavior

- It safely skips broken or incomplete components.
- It logs errors if something is not usable.

4. Create GameObject With Mesh

Purpose

Builds a clean standalone object from extracted mesh data.

What it does

Creates a new object and attaches:

- A MeshFilter → holds the geometry
- A MeshRenderer → handles materials

Then:

- Assigns the mesh to the filter
- Assigns materials to the renderer

Result

A fully functional renderable object that matches the original mesh visually.

Usage

Internally used by the conversion function for every valid mesh found.

5. Create Prefab

Purpose

Takes all generated GameObjects and builds a prefab asset.

What it does

1. Validates that objects exist.
2. Creates a root object:
 - If only one object → it becomes the prefab root
 - If multiple objects → creates a parent container
3. Builds prefab file path:
 - Uses the same folder as the original model
 - Names it: "ModelName (Prefab)"
4. Saves prefab using Unity's asset system.
5. Cleans up temporary objects from the scene.

Safety features

- Wrapped in error handling (try/catch).
- Logs success or failure.
- Always destroys temporary objects afterward.

Variables (Conceptual Explanation)

selectedObjects

Represents everything currently selected in the Unity Project window.

Used as the input source for the entire process.
models A filtered list containing only valid 3D model assets. Used to ensure the script only processes real model files.
createdObjects Stores all GameObjects generated from mesh extraction. Used later to build the prefab structure.
mesh / materials (temporary variables) Hold extracted rendering data from components. Used to rebuild the visual representation of the model.
prefabRoot The final root object that becomes the prefab. It is either: • A single mesh object, or • A parent containing multiple mesh objects
prefabPath The file location where the prefab will be saved. Derived from the original model's folder.
Usage Summary (Practical) To use this tool: 1. Select one or more 3D models in Unity (FBX, etc.). 2. Right-click or use the top menu: ◦ Assets → Tools → MMD Collection → Create Prefabs From Selected Model 3. Wait for processing. 4. A new prefab appears in the same folder as the model.
What this script is useful for • Breaking complex models into clean prefab structures • Extracting meshes from imported assets • Rebuilding assets for custom pipelines • Debugging or reorganizing model hierarchies • Preparing models for modular systems

Find Missing Scripts
This script is a Unity Editor utility that scans the entire currently loaded scene and detects GameObjects that have missing script references (i.e., broken or removed MonoBehaviour components). It then logs the result and automatically selects those objects in the editor so you can inspect or fix them.
Overall Purpose In Unity, a “missing script” appears when: • A script was deleted or renamed • A prefab or scene still references the old component • Unity cannot resolve the component type anymore This tool helps by: • Finding all such broken objects in the scene • Listing them in the console • Selecting them in the Hierarchy for quick fixing
How the Script Works (High Level Flow) 1. User clicks a menu item in Unity. 2. Script scans all loaded GameObjects.

3. It filters out invalid or hidden objects.
4. It checks each object's components.
5. If any component is missing (null), the object is marked.
6. All results are:
 - Logged in the console
 - Selected in the editor

Methods Explained

Main Method: Find Missing Scripts

Purpose

This is the entry point executed from Unity's menu:

- GameObject → Tools → MMD Collection → Find Missing Scripts

What it does step-by-step

1. Creates a storage list

A list is created to store all GameObjects that contain missing scripts.

- Initially empty
- Filled only when problems are found

2. Collects all GameObjects

It searches the entire editor environment and gathers:

- All GameObjects in loaded scenes
- Including inactive objects

This is a broad scan of everything currently open in Unity.

3. Filters invalid objects

Each object is checked and skipped if:

- It is not part of a valid scene
- The scene is not loaded
- The object is hidden or marked as non-editable

This prevents false positives from editor-only or internal objects.

4. Reads components

For each valid GameObject:

- It retrieves all attached components (scripts, renderers, colliders, etc.)

5. Detects missing scripts

A missing script in Unity appears as:

- A component entry that is "null"

So the logic checks:

- If any component equals null → that object has a missing script

If true:

- The GameObject is added to the result list

6. Logs result

After scanning everything:

- It prints how many objects were found with missing scripts

This helps confirm whether cleanup is needed.

7. Selects objects in Unity

Finally:

- All detected objects are automatically selected in the editor

This allows you to:

- Inspect them in the Hierarchy
- Fix or remove broken references quickly

Variables Explained

objectsWithMissingScripts

This is the result container.

It stores every `GameObject` that contains at least one missing script.

Used at the end for:

- Logging
- Selection in Unity Editor

allObjects

This holds every `GameObject` currently loaded in the editor.

It includes:

- Scene objects
- Prefab instances in memory
- Possibly hidden or editor-only objects (filtered later)

obj (loop variable)

Represents each `GameObject` being checked one by one during the scan.

components

This is the full list of components attached to a single `GameObject`.

It may include:

- Scripts
- Renderers
- Colliders
- Audio sources
- Etc.

It is used to detect broken script references.

Key Logic Concept

The most important idea in this script is:

In Unity, a missing script is represented as a null component entry.

So instead of checking script names or types, the system simply checks:

- “Does this object have any component that is null?”

If yes → broken script detected.

Usage (How You Use It)

1. Open Unity Editor.
2. Load the scene you want to clean.
3. Go to:

`GameObject` → Tools → MMD Collection → Find Missing Scripts

4. After execution:
 - Console shows number of broken objects
 - Those objects become selected in the Hierarchy

Why This Script Is Useful

- Helps clean broken prefabs and scenes
- Speeds up debugging large projects
- Prevents runtime errors caused by missing scripts
- Useful after refactoring or renaming scripts

Paste As Child Multiple

This script is a **Unity Editor window tool** that lets you take a single prefab or scene object and **duplicate it as a child under multiple selected GameObjects at once**. It also supports undo, optional name numbering, and improved editor feedback.

Overall Purpose

Normally in Unity you can:

- Paste or duplicate an object under one parent at a time

This tool extends that by allowing you to:

- Select many parent objects
- Paste the same object under all of them in one action

- Automatically organize and optionally rename each copy

How the Tool Works (High Level Flow)

1. User opens the editor window from a menu.
2. User selects an object to copy.
3. User selects one or more parent objects in the hierarchy.
4. User clicks “Paste As Child”.
5. Script:
 - Instantiates copies of the object
 - Parents each copy under each selected object
 - Resets transforms
 - Optionally renames them
 - Registers undo
 - Selects results in the editor

Editor Window Fields

1. Object to Copy

This is the main input of the tool.

It stores:

- A prefab OR
- A scene object

This is the object that will be duplicated.

Usage

You drag or select any GameObject into this field before running the tool.

2. Enumerate Names

A boolean toggle that controls naming behavior.

If enabled:

Each created object gets a numbered suffix like:

- Object (1)
- Object (2)
- Object (3)

If disabled:

All copies keep the same original name.

Window Setup Method

Init (Menu Entry)

Purpose

Creates and opens the custom editor window.

Trigger

Accessible from Unity menu:

- GameObject → Tools → MMD Collection → Paste as Multiple Children

What it does

- Opens a fixed-size editor window
- Sets title: “Paste as Multiple Children”
- Prevents resizing beyond defined limits

OnGUI (Interface Layout)

This controls everything visible in the editor window.

What it displays

1. Title label (“Select Prefab or Scene Object to Copy”)
2. Object selection field
3. Enumerate toggle
4. Button to execute paste

Button Behavior

The “Paste As Child” button is:

- Disabled if no object is selected
- Enabled only when a valid object exists

When clicked → triggers the main logic function.
Core Method: Paste As Child This is the main engine of the tool.
Step 1: Get selected parents It reads all currently selected objects in the hierarchy. These become the parents for the new copies.
Step 2: Validation checks It ensures: • At least one parent is selected • A valid object is assigned If not: • It prints warnings and stops
Step 3: Setup tracking It creates: • A list to store all created objects • A counter for numbering (if enabled) Also: • Starts an Undo group (so all actions can be undone together)
Step 4: Loop through all parents For each selected parent: A. Create a copy of the object Two possible cases: • If it is a prefab asset: → It uses prefab instantiation (safe prefab cloning) • If it is a scene object: → It uses normal duplication This ensures correct behavior in both cases.
B. Register undo Each created object is registered so the action can be undone.
C. Parent the object The new object is: • Attached to the selected parent • Reset so it aligns perfectly: ◦ Position → (0,0,0) ◦ Rotation → default ◦ Scale → 1,1,1 So every copy starts clean inside its parent.
D. Optional naming If enumeration is enabled: • Each object gets a unique number suffix This avoids confusion when many copies are created.
E. UX improvements For each creation: • The parent object is selected • The new object is highlighted in the editor This helps visually confirm what was created.
F. Error handling If anything fails: • The error is caught

• A message is printed instead of crashing the tool
Step 5: Final selection After processing all parents: • All created objects are selected at once So the user immediately sees results.
Step 6: Undo cleanup The undo operations are grouped together: • So one undo press reverses everything at once
Step 7: Close window Once finished: • The editor window automatically closes This makes the tool behave like a quick action panel.
Variables Explained objectToCopy Holds the original object being duplicated. Can be: • Prefab asset • Scene object This is the source for all clones.
enumerate Controls whether objects get numbered names. • True → unique names • False → identical names
parents Stores all currently selected GameObjects in the hierarchy. These become the targets for parenting.
createdObjects Stores all newly created copies so they can: • Be selected at the end • Be grouped for undo • Be tracked during execution
globalCount A simple counter used for naming. It increases every time a new object is created.
undoGroup Tracks the undo session so all actions can be collapsed into one step.
Usage (Practical) How to use the tool: 1. Open Unity. 2. Go to: ◦ GameObject → Tools → MMD Collection → Paste as Multiple Children 3. Select an object to copy. 4. Select one or more parent objects in the hierarchy. 5. Click “Paste As Child”. 6. All selected parents receive a copy as a child.
Why This Tool Is Useful • Speeds up level building • Useful for placing repeated objects across many parents

- Reduces repetitive drag-and-drop work
- Keeps hierarchy clean and structured
- Fully supports undo for safety

Material Property Cleaner

This script is a **Unity Editor cleanup utility** designed to fix broken or outdated material data. It scans selected materials and removes anything that no longer matches the shader they use, including invalid saved properties and obsolete shader keywords.

Overall Purpose

Over time, Unity materials can accumulate:

- Properties that no longer exist in the shader
- Old values from previous shader versions
- Invalid or leftover shader keywords

This tool:

- Validates materials against their current shader
- Deletes unsupported properties
- Removes invalid keywords
- Saves the cleaned result back to the asset database

How the Script Works (High Level Flow)

1. User selects one or more materials in Unity.
2. User runs a menu command.
3. Script confirms the operation.
4. For each material:
 - Collect valid shader properties
 - Remove invalid saved properties
 - Clean invalid shader keywords
 - Save changes to disk

Main Entry Method

Clean Material Properties

Purpose

This is the main function triggered from Unity's menu:

- Assets → Tools → MMD Collection → Clean Invalid Material Properties

Step-by-step behavior

1. Confirmation dialog

Before doing anything, the script asks the user:

- "Are you sure?"

If the user declines → the operation stops immediately.

This prevents accidental data loss.

2. Loop through selected objects

The script processes everything currently selected in the Project window.

For each item:

- If it is a Material → it is processed
- If not → a warning is logged

3. Validate shader existence

For each material:

- It checks if a shader is assigned
- If not → it logs an error and skips it

Because without a shader, validation is impossible.

4. Register Undo

Before modifying anything:

- The material is registered in Unity's Undo system

This allows the user to revert changes.

5. Build list of valid shader properties

The script queries the shader and collects all valid property names such as:

- floats
- textures
- integers
- colors

This becomes the “reference list” used for validation.

6. Access internal material data

Unity stores material values in a hidden structure called:

- Saved Properties

The script reads this internal data using serialization.

It extracts:

- Texture properties
- Integer properties
- Float properties
- Color properties

7. Remove invalid properties

Each category is cleaned:

- If a property name does not exist in the shader → it is deleted

This ensures the material only keeps valid data.

8. Apply changes

After cleanup:

- The modified serialized data is written back to the material

9. Clean shader keywords

The script then removes invalid shader keywords:

- Keywords that are not supported by the current shader are disabled

This prevents rendering issues or broken shader states.

10. Save and refresh

Finally:

- The material is marked as modified
- Unity saves changes to disk
- The asset database is refreshed

Utility Methods

1. Remove Invalid Properties

Purpose

Cleans arrays of saved material properties.

What it does

For each property list (textures, floats, ints, colors):

- It loops through the list backwards (important for safe deletion)
- Checks if each property name exists in the valid shader property set
- If not → removes it

Why reverse looping is used

Because removing items while looping forward would:

- Shift indices
- Cause skipped elements or errors

2. Clean Invalid Keywords

Purpose

Removes shader keywords that no longer belong to the shader.
Step-by-step behavior 1. Ensure shader and material are valid If either is missing → nothing happens.
2. Cache reflection method The script uses internal Unity functionality to access: - global shader keywords This is not normally public, so it uses reflection. To avoid repeated expensive lookups: - the method is cached after first use
3. Get valid keywords It retrieves all valid keywords supported by the shader. These are stored in a lookup set for fast checking.
4. Compare against material keywords It loops through all keywords currently on the material: - If keyword is not in the valid set: - It is disabled - It is added to a “removed list”
5. Log removed keywords If any were removed: It prints a list of cleaned keywords for debugging
Variables Explained
getShaderGlobalKeywordsMethod A cached reflection reference to Unity’s internal function that retrieves shader keywords. Why it exists - Avoids repeated expensive reflection calls - Only initialized once
validProperties A set containing all valid shader property names. Used to quickly check if a material property is still valid.
matSO (SerializedObject) A Unity editor representation of the material’s internal data. Used to safely modify hidden material fields.
savedProps The internal container holding all serialized material properties. Includes textures, floats, colors, and ints.
validKeywords A set of all keywords supported by the shader. Used for fast lookup when cleaning keywords.
removedKeywords Stores keywords that were removed during cleanup. Used only for logging purposes.
Usage (How You Use It) Step-by-step - Open Unity Editor - Select one or more materials in the Project window

3. Run:
Assets → Tools → MMD Collection → Clean Invalid Material Properties
4. Confirm the warning dialog
5. The script:
 - Cleans invalid properties
 - Removes outdated shader keywords
 - Saves updated materials automatically

Why This Tool Is Useful

- Fixes broken materials after shader updates
- Removes leftover garbage data from old pipelines
- Prevents rendering inconsistencies
- Keeps project assets clean and optimized
- Helps when migrating between shader versions

Material Shader Converter

This script is a Unity Editor tool designed to convert a large set of legacy **MMD4Mecanim materials** into newer **URP-compatible materials**, while preserving as much visual fidelity as possible. It does this by mapping old shader names to new ones, transferring material properties, and cleaning up invalid data after conversion.

Core Idea of the Tool

When you run the menu command in Unity's Project window:

Assets → Tools → MMD Collection → Convert Material Shader (MMD4Mecanim)
the script:

1. Reads all selected materials.
2. Checks if their shader exists in a predefined conversion map.
3. Converts them into a new URP shader.
4. Transfers old visual settings (color, outline, transparency, etc.).
5. Fixes render settings and removes invalid leftovers.

1. ShaderModel (internal classification system)

This is an internal "label system" used to decide how a material should be converted.

Each value represents a different rendering style:

- **Default** → normal URP shader conversion.
- **Tessellation** → shader supports mesh surface subdivision (smoother geometry).
- **Empty** → minimal dummy shader (no real rendering logic).
- **FourLayers / EightLayers** → multi-outline system (used for anime-style outlines).
- **NoShadow** → disables shadow casting.
- **NoShadowAndTessellation** → combination of both behaviors.

👉 This allows the tool to apply different conversion logic depending on the material type.

2. Shader Mapping Dictionary (conversion table)

This is the heart of the tool.

It maps:

- Old shader name (MMD4Mecanim)
→ to
- New URP shader name + configuration rules

Each entry contains:

- **new shader path** → where to switch the material
- **transparent flag** → whether the material becomes transparent
- **outline flag** → whether outlines are enabled
- **twoSide flag** → double-sided rendering
- **gShad flag** → global shadow control
- **ShaderModel type** → which conversion logic to use

👉 Example idea:

- A "transparent + edge + both faces" shader becomes a URP shader with transparency enabled, outline enabled, and double-sided rendering turned on.

3. Menu Entry: ConvertShader()

This is the function you call from Unity.

What it does:

- Loops through all selected assets.
- Filters only materials.
- Checks if the material's shader exists in the mapping table.
- If found → sends it to conversion logic.
- If not found → logs a warning.

👉 This is the entry point of the tool.

4. ApplyShaderConversion()

This is the main “controller” of the conversion process.

It does 5 important things:

1. Undo support

It registers the material so changes can be undone.

2. Saves important settings

Before changing shader, it stores:

- instancing settings
- GI settings
- render flags

So nothing critical is lost.

3. Chooses conversion type

Depending on ShaderModel:

- Default / Tessellation / NoShadow → standard conversion
- Empty → minimal shader setup
- FourLayers / EightLayers → multi-outline conversion

4. Restores render settings

After shader swap, it restores:

- instancing
- GI behavior

5. Cleans leftover data

Removes:

- invalid material properties
- invalid shader keywords

5. ApplyStandard()

This is the most complex function.

It handles full-feature materials like lit shaders.

Step-by-step behavior:

A. Reads old material data

It extracts:

- Base color
- Specular color
- Ambient color
- shininess
- textures (main, toon, sphere)
- edge/outline settings
- shadow flags
- render queue
- tessellation values (if needed)

👉 This is done so nothing visually breaks after conversion.

B. Switches shader

It replaces the old shader with the new URP shader.

C. Restores visual appearance

It re-applies:

- colors
- textures
- toon shading settings
- specular settings

Even if property names differ, it tries to preserve meaning.

D. Configures rendering behavior

- culling (two-sided or not)
- shadow casting
- self-shadow
- receive shadows

E. Handles outline system

If outline is enabled:

- sets outline color
- sets outline thickness
- adjusts outline transparency rules

F. Handles transparency

If material is transparent:

- switches render mode
- adjusts render queue

Otherwise:

- sets opaque mode

G. Handles special modes

If shader is NoShadow:

- disables shadow casting completely

If Tessellation:

- restores tessellation parameters

6. ApplyEmpty()

Used for dummy or placeholder materials.

It:

- assigns minimal shader
- forces transparent-like rendering
- disables shadows
- ensures clean render state

👉 Think of it as “safe fallback material”.

7. ApplyMultiplePass()

Used for outline-heavy shaders.

It:

- switches shader
- restores edge color and size
- sets number of outline layers:
 - 0 → 4 layers
 - 1 → 8 layers
- keeps render queue unchanged

👉 This supports stylized anime outline systems.

8. RenderQueueToTransparent()

A helper that adjusts render order.

If a material is opaque, but becomes transparent:

- it shifts render queue into a transparent range

This ensures correct rendering order in Unity.

9. CleanMaterialProperties()

After shader change, old properties may no longer exist.

This function:

1. Gets valid shader properties

It queries the shader for all valid property names.

2. Accesses saved material data

Unity stores old values in internal structure:

- textures
- floats
- colors
- ints

3. Removes invalid entries

Anything not in the shader is deleted.

4. Cleans keywords

Calls keyword cleanup function to remove unused shader flags.

10. RemoveInvalidProperties()

A low-level cleanup function.

It:

- iterates backwards through property list
- checks if property name is valid
- deletes it if not valid

👉 Backwards iteration avoids index shifting bugs.

11. CleanInvalidKeywords()

Removes broken shader keywords.

How it works:

- Uses reflection to access Unity's internal shader keyword list
- Gets valid keywords for current shader
- Compares against material keywords
- Disables invalid ones

👉 This prevents broken rendering states.

Final Summary (What the tool achieves)

This system:

- Converts old MMD shaders → URP shaders
- Preserves material appearance as much as possible
- Handles transparency, outlines, and tessellation
- Fixes broken shader data automatically
- Keeps Unity project clean and compatible

Header Group GUI

This script is a small **Unity Editor UI utility** that creates collapsible (foldout) sections in custom inspectors and editor windows, while also remembering whether each section is open or closed between sessions.

It is essentially a reusable “UI building block” for cleaner inspector layouts.

Overall Purpose

When you call this utility, it draws:

- A styled header bar
- A clickable foldout arrow
- A collapsible content area
- Persistent open/closed state (saved in Unity preferences)

So even after closing Unity, the same sections stay expanded or collapsed exactly as before.

Core Storage System (State Cache)

foldoutStates (dictionary cache)

This is an in-memory storage that maps:

- group name → open/closed state (true/false)

Why it exists:

Without it, Unity would constantly read from disk (EditorPrefs), which is slower.

So this acts as a **fast lookup cache**.

Public Function: DrawGroup

This is the main function you call to create a collapsible section.

Inputs:

- **groupName**
 - The label shown in the inspector header
 - Also used as the unique ID for saving state
- **drawContent**
 - A callback function
 - Contains everything that should appear inside the foldout

👉 Think of it as:

“Draw this header, and if open, draw this content.”

Step-by-step behavior

1. Load saved state

It asks:

“Is this group currently open or closed?”

This comes from:

- memory cache first
- then EditorPrefs (persistent storage)

2. Cache layout values

It stores UI metrics like:

- line height (standard inspector row height)
- full inspector width

This avoids repeated Unity API calls.

3. Create header rectangle

It reserves a space in the inspector for the header.

Then it forces:

- full width alignment
- consistent positioning

4. Draw background visuals

Two visual layers are drawn:

- a thin dark top border (separation line)
- a semi-transparent background bar

👉 This makes the foldout visually distinct from other inspector elements.

5. Adjust layout for foldout icon

The rectangle is shifted right to make room for:

- arrow icon
- indentation space

6. Define visual style

It uses Unity’s built-in foldout style but modifies it:

- makes text bold

So the header stands out.

7. Create label

A label is built using:

- text (group name)

- tooltip (optional hover help text)

8. Draw the foldout

This is the interactive part:

- user clicks arrow or label
- state changes (open/closed)

The result is stored in:

- newState

9. Save state if changed

If the user toggled it:

- update in-memory cache
- save to EditorPrefs (persistent storage)

So it survives Unity restart.

10. Draw content (if open)

If the group is expanded:

- increase indentation level
 - makes content visually nested
- execute drawContent callback
- restore indentation
- add spacing at the bottom

👉 This is where custom inspector UI is rendered.

Internal Function: GetGroupState

This retrieves whether a group is open or closed.

Logic:

1. Check memory cache first
2. If not found:
 - read from EditorPrefs (disk storage)
 - default = true (open by default)
3. Store result in cache
4. Return value

👉 This ensures:

- fast access after first load
- persistence across Unity sessions

Internal Function: SaveGroupState

This updates the foldout state.

It does two things:

- updates memory cache
- writes to EditorPrefs (persistent storage)

So the state is:

- instantly usable
- permanently saved

Internal Function: GetPrefsKey

This generates a unique key used in Unity's EditorPrefs system.

Format:

- "HeaderGroupGUI_Foldout_" + group name

👉 Example:

- "HeaderGroupGUI_Foldout_Transform Settings"

This prevents conflicts between different groups.

Usage Example (conceptual)

You would use it like:

- call DrawGroup with a label like "Settings"
- pass a function that draws fields inside it

Result:

- collapsible inspector section
- persistent state
- clean UI grouping

Key Design Idea

This system is optimized around:

1. Performance

- avoids repeated EditorPrefs calls via caching

2. Clean UI

- consistent header styling
- automatic indentation handling

3. Persistence

- remembers foldout state across Unity sessions

Summary

This script creates a reusable inspector feature that:

- draws collapsible sections
- remembers open/closed state
- caches data for performance
- provides clean, styled editor UI
- supports any custom inspector content via callbacks

Material Utility

This script is a **Unity Editor utility library for custom material inspectors**, designed to standardize and simplify how shader properties, surface settings, and rendering options are displayed and edited inside the Inspector.

It is not gameplay code. It only runs inside the Unity Editor and focuses on **UI drawing, material editing, and editor-state synchronization**.

Below is a structured explanation of how it works, covering **state, methods, data flow, and usage**.

1. Overall Purpose

This utility acts like a **toolbox for custom material inspectors**, especially for MMD (MikuMikuDance-style) materials.

It provides:

- Custom UI controls (sliders, toggles, dropdowns, color pickers)
- Surface and blending configuration (opaque / transparent)
- Texture handling (including tiling/offset)
- Shader keyword and render pass synchronization
- Safe multi-object editing support
- Undo support for every modification

Instead of writing inspector logic repeatedly, other scripts call these methods.

2. Core Concept: How the Script Works

Every method follows the same pattern:

1. **Get material properties** from selected materials
2. **Detect mixed values** (when multiple materials differ)
3. **Draw a Unity Inspector UI control**
4. **Detect user changes**
5. **Apply changes to all selected materials**
6. **Register Undo + mark materials dirty**

This makes it safe for multi-selection editing in Unity.

3. Data Handling (Custom MMD Data System)

LoadData

Purpose

Reads saved metadata from a custom container into editor variables.

Inputs

- A data container holding MMD metadata

- A target material
- Outputs:
 - Japanese name
 - English name
 - memo
 - flag indicating default inspector usage

How it works

- Starts with safe empty values
- If data or material is missing → exits immediately
- Reads a global flag (showSystemsDefault)
- Searches a list of stored material entries
- If a matching material is found:
 - Copies stored metadata into outputs

Usage

Used when opening a material inspector to **restore saved metadata UI values**.

SaveData

Purpose

Writes metadata back into the MMD data container.

Inputs

- Data container
- Material
- Names (JP/EN)
- Memo
- Default inspector flag

How it works

- Updates global inspector flag
- Searches if material already exists in list
- If found → updates fields
- If not found → creates a new entry

Usage

Used when user edits metadata in inspector and saves it.

4. Global Illumination UI

RenderLightmapFlags

Purpose

Draws a dropdown controlling how a material interacts with Unity Global Illumination.

Key variables

- displayOptions: text shown in UI
- flagOptions: actual Unity enum values
- firstValue: reference material value
- hasMixedValue: detects multi-material mismatch

How it works

- Checks if selected materials differ → shows “mixed state”
- Displays dropdown
- If changed:
 - Applies same value to all materials
 - Uses Undo system
 - Marks materials dirty

Usage

Used in custom material inspectors for lighting behavior control.

5. Property Drawers (Core UI System)

These are reusable UI blocks for shader properties.

ColorProperty

Purpose

Draws a color picker + manual RGBA fields.

Features

- HDR support
- Alpha toggle
- Per-channel editing
- Mixed value support

Flow

- Get color property from materials
- Draw color field
- If changed → update material
- Also draws numeric RGBA fields
- Syncs both UI styles together

Usage

For shader color properties like albedo or emission.

FloatProperty / FloatSliderProperty

Purpose

- FloatProperty → numeric input
- FloatSliderProperty → slider control

Flow

- Get float property
- Draw UI
- Detect change
- Apply to all materials

Usage

- Roughness, metallic, intensity, etc.

VectorProperty

Purpose

Edits Vector2/3/4 shader values.

Flow

- Clamp dimension (2–4)
- Extract vector
- Draw X/Y/Z/W fields depending on dimension
- Apply updated vector

Usage

- UV scale, direction vectors, offsets

6. Toggle & Keyword System

FloatToggleProperty

Purpose

Treats a float as a boolean (0 or 1).

Usage

- shader switches stored as float values

PassToggleProperty

Purpose

Enables/disables shader passes.

Key idea

Each material can enable or disable rendering passes dynamically.

KeywordToggleProperty

Purpose

Synchronizes:

- float property
- shader keyword

Flow

- Toggle UI
- Update float value
- Enable/disable keyword per material

Usage

Used for shader features like:

- normal maps
- receive shadows
- special effects toggles

7. Dropdown Systems

FloatDropdownProperty

Purpose

Simulates enum behavior using float values.

Flow

- Convert float → index
- Show dropdown
- Convert back → float

SurfaceTypeDropdownProperty

Purpose

Switch between:

- Opaque
- Transparent

Important behavior

When changed:

- Updates `_Surface`
- Updates `RenderType` tag
- Adjusts render queue

Usage

Core surface mode selector.

BlendingModeDropdownProperty

Purpose

Controls how transparent materials blend.

Internals

Sets:

- blend mode
- source blend factor
- destination blend factor

Usage

Used only for transparent materials.

DepthWriteDropdownProperty

Purpose

Controls ZWrite behavior:

- Auto
- Force Enabled
- Force Disabled

Also:

- toggles `DepthOnly` pass

8. Texture System

TextureProperty

Purpose

Handles:

- texture selection
- cubemap restriction
- tiling and offset UI

Flow

1. Draw texture field
2. Apply texture to materials
3. Optionally show scale/offset UI

DrawScaleOffsetInRect

Purpose

Draws:

- tiling (scale)
- offset

inside a custom UI layout.

Flow

- Detect mixed values
- Draw Vector2 fields
- Apply per material

9. Surface Options Composite UI

DrawSurfaceOptions

Purpose

This is the **main combined inspector block**.

It calls multiple systems:

- Surface type
- Blending mode (only if transparent)
- Render face (`_Cull`)
- Depth write
- Depth test
- Alpha clipping
- Shadow casting
- Shadow receiving

Key logic

- Some controls appear conditionally
 - example: blending only if transparent
 - alpha cutoff only if clipping enabled

Usage

This is likely the main function called in a custom material inspector.

10. Render Queue Conversion

ConvertRenderQueueToTransparent / Opaque

Purpose

Automatically shifts render queue values depending on surface mode.

Logic

- Geometry range → shifted to Transparent range
- AlphaTest range → slightly adjusted
- Reverse conversion restores original behavior

Usage

Prevents rendering order bugs when switching surface types.

11. Validation Helpers

HasFloatPropertyValue

Checks:

- if all materials share same value
- if that value matches expected

Used for conditional UI display.

CheckBlendingMode

Purpose

Ensures blending settings are always valid.

Flow

- Reads surface + blend mode
- Calculates correct src/dst blend values
- Applies automatically

Usage

Called to keep material state consistent.

12. Key Design Patterns

This script consistently uses:

1. Multi-object editing

All methods support editing multiple materials at once.

2. Mixed-value detection

Shows Unity-style “—” when values differ.

3. Undo safety

Every modification is wrapped in Undo recording.

4. Editor-only logic

Everything runs only in Unity Editor (not runtime).

5. Centralized UI system

Instead of repeating inspector code, everything is reusable.

13. How to Use This Script (Practical)

In a custom material inspector:

You typically call:

- DrawSurfaceOptions(materialProperties)
- TextureProperty(...)
- ColorProperty(...)

Example usage conceptually:

- Create custom inspector
- Pass shader properties array
- Call these utilities to draw UI blocks

14. Summary

This script is essentially a:

Full modular material inspector framework for Unity Editor

It provides:

- A complete surface/shader editing system
- Standardized UI components
- Automatic rendering state management
- Safe multi-material editing
- Built-in synchronization between UI → material → shader

MMD Material Custom Inspector Base

This script is a **base framework for custom Unity material inspectors (ShaderGUI)** specifically designed for MMD-style materials. It does not define a single inspector by itself — instead, it provides a **shared system that all custom MMD material inspectors inherit from**.

Its main job is to handle:

- Shared UI structure (headers, memo, names)
- Multi-material editing
- Undo/Redo safety
- Loading/saving metadata (CustomMMDData)
- Delegating shader-specific UI to child classes
- Switching between custom UI and Unity default inspector

Below is a full breakdown of **variables, methods, and usage flow**.

1. Core Role of the Class

This class is a **foundation layer**:

Any custom material inspector in the project inherits from it and automatically gets:

- Standard MMD header UI
- Name + memo system
- Default/custom toggle mode
- Undo-safe editing
- Shared rendering rules
- Hooks for shader-specific UI

Think of it as:

“A reusable inspector engine for all MMD materials”

2. Core Variables (State)

materialEditor

Represents Unity's internal material inspector controller.

Purpose:

- Draw default Unity UI fields
- Access built-in inspector functions (render queue, instancing, etc.)

Usage:

Used whenever the script needs Unity's built-in material tools.

properties

Array of all shader properties in the current material.

Purpose:

- Contains everything from shader: floats, textures, vectors, toggles

Usage:

Passed into all utility methods to draw custom controls.

targets

All selected objects in the inspector (usually materials).

Purpose:

- Enables multi-material editing

Usage:

If multiple materials are selected, every change applies to all of them.

3. MMD Custom Data System

These fields store metadata separate from the shader:

mmdData

Shared data container (ScriptableObject)

Stores:

- Japanese name
- English name
- memo text
- inspector mode flag

nameJP / nameEN / memo

Cached values of the selected material metadata.

Purpose:

- Display in inspector UI
- Avoid repeated loading from data asset

showDefault

Boolean flag controlling UI mode:

- true → show Unity default inspector
- false → show custom MMD inspector

4. Main Entry Point

OnGUI

This is the **core inspector function**.

What it does step-by-step:

1. Cache Unity context

Stores:

- materialEditor
- properties
- targets

2. Load required systems

- Ensures MMD data exists
- Ensures Undo system is registered

3. Begin change tracking

Starts monitoring if user modifies anything in UI.

4. Draw header

Always shown:

- MMD Data section

Includes:

- names
- memo
- toggle for default mode

5. UI mode switch

If `showDefault = false` (custom mode):

- Draw custom shader UI
- Draw surface options (opaque/transparent, etc.)
- Draw advanced rendering options:
 - render queue
 - instancing
 - GI flags

If `showDefault = true`:

- Skip custom UI
- Show Unity default inspector instead

6. Apply rules (only if something changed)

When user modifies anything:

- Synchronizes blending mode
- Applies shared rules
- Applies shader-specific rules

5. Undo / Redo System

EnsureUndoHook

Purpose

Registers a callback so the inspector reacts to Undo/Redo.

Behavior:

- Removes old hook (prevents duplicates)
- Adds new hook

OnUndoRedo

Triggered when user presses Undo/Redo.

What it does:

- Clears cached data
- Forces inspector repaint
- Forces scene refresh

Why:

Keeps UI and data consistent after undo operations.

6. Header UI System

DrawHeaderUI

This is the top section of the inspector.

Contains:

1. Show Default Systems toggle

Switches between:

- custom inspector
- Unity inspector

When changed:

- Saves state into MMD data for each material
- Updates cached value

2. Material names

Two fields:

- Japanese name
- English name

Uses mixed-value detection for multi-selection.

7. Memo System

DrawMemoData

Purpose

Displays and edits a multi-line memo stored in MMD data.

Features:

- Supports multiple materials
- Detects mismatched values
- Undo-safe editing

Flow:

1. Compare memo across all materials
2. If different → show “mixed value”
3. User edits text
4. Save to all materials via MMD data

8. Mixed String Field System

DrawMixedMaterialStringField

Purpose

Reusable field for:

- Japanese name OR English name

Inputs:

- label (UI text)
- isJP (which field is being edited)
- cachedValue
- width

Behavior:

1. Detects if materials have different values
2. Shows mixed-state UI if needed
3. Draws text field
4. Applies changes to all selected materials

Usage:

Used for:

- Mat-Name (JP)
- Mat-Name (EN)

9. Custom UI Hooks

DrawUI

Purpose

Empty method meant to be overridden.

Usage:

Child inspectors implement shader-specific UI here.

Example:

- toon shader controls
- outline settings
- special effects

10. Rendering Rule Hooks

ApplyCommonRules

Purpose

Shared logic applied to all MMD materials.

ApplyShaderSpecificRules

Purpose

Specialized logic for derived shaders.

Why two layers?

Separates:

- universal behavior
- shader-specific behavior

11. Custom Data Loading System

EnsureCustomDataLoaded

Purpose

Initializes MMD data system once per inspector session.

Flow:

1. If already loaded → do nothing
2. Load or create MMD data asset
3. Remove invalid references
4. Load values from first selected material:
 - JP name
 - EN name
 - memo
 - default mode flag

12. Full Inspector Flow (How It All Works Together)

When Unity draws a material inspector:

Step 1

OnGUI is called

Step 2

System initializes:

- editor reference
- properties
- targets
- MMD data

Step 3

Header is always drawn:

- names
- memo
- default toggle

Step 4

Depending on mode:

- custom UI OR Unity default UI

Step 5

User edits values

Step 6

If changes happen:

- blending rules updated
- shader rules applied
- data saved safely

Step 7

Undo system ensures reversibility

13. Key Design Ideas

This system is built around:

1. Inheritance-based inspector system

All shaders extend this base.

2. Data separation

- shader data (Material)
- metadata (CustomMMDData)

3. Multi-material safety

Every field supports multiple selected materials.

4. Editor stability

- Undo-safe
- mixed-value UI
- repaint handling

5. Flexible UI mode switching

Custom vs Unity default inspector

14. Usage Summary

To use this system:

1. Create a new material inspector class
2. Inherit from this base class
3. Override:
 - DrawUI (custom controls)
 - ApplyShaderSpecificRules (optional)
4. Unity automatically uses:
 - header system
 - memo system
 - rendering controls
 - undo handling

15. Final Concept

In simple terms:

This script turns Unity's ShaderGUI into a **modular inspector framework with metadata, UI layers, and rendering rule automation**

MMD Material SG / MMD Material ASE

This script is a **custom Unity material inspector** for an MMD (MikuMikuDance) Shader Graph shader. It defines how the material UI is organized inside the Unity Inspector, how values are edited, and how those values automatically control rendering behavior like transparency, shadows, blending, and shader keywords.

It inherits from a shared base inspector system, so it only focuses on **what this specific shader shows and how it behaves**, while the base class handles common infrastructure.

Overall Purpose

This class:

- Builds a **custom inspector UI** divided into sections (color, rendering, textures, effects, etc.)
- Lets you edit material properties in a structured way
- Automatically synchronizes:
 - shader floats (like opacity, shininess, etc.)
 - shader keywords (like shadows, transparency, effects)
 - rendering rules (like blending and alpha clipping)
- Applies extra logic when values change (like enabling/disabling effects)

Class Structure

Inheritance (base behavior)

This inspector inherits from a base system that already provides:

- Multi-material editing support
- Undo/Redo safety
- Header UI system
- Memo + metadata storage (MMD-specific data)
- Utility access (color fields, sliders, dropdowns, etc.)

So this class only defines:

- UI layout
- shader-specific behavior rules

Main Inspector Method (core UI)

DrawUI (main layout function)

This is the **central function that builds the entire inspector UI**.
It is divided into sections:

1. Material Color section

Controls visual appearance of the material:

- Diffuse color → base color of the material
- Specular color → reflection highlights
- Ambient color → lighting influence
- Opaque slider → transparency control
- Reflection value → shininess intensity

 Usage:

This section is used when artists want to adjust how the material looks under lighting.

2. Rendering section

Controls how the material interacts with the GPU rendering pipeline:

Includes:

- Double-sided rendering toggle
 - Makes mesh visible from both sides
- Shadow casting toggle
 - Controls whether object casts shadows
- Shadow receiving toggle
 - Controls whether object receives shadows
- Self-shadowing option
 - Special internal shading behavior

 Usage:

Used to define how the object behaves in the scene lighting system.

3. Edge (Outline) section

Controls outline effects:

- On/Off toggle (currently disabled visually)
- Outline size
- Outline color (with alpha transparency)

 Usage:

Used for toon-style outlines around characters.

4. Texture / Memo section

Controls textures and special effects:

- Effects dropdown:
 - Disabled
 - Multi-sphere reflection
 - Additive sphere reflection
 - Sub-texture mode
- Main texture
- Toon texture
- Conditional logic:
 - If Sub-Tex mode is active:
 - UV layer selection appears
 - Sub texture field appears
 - Otherwise:
 - Cubemap reflection texture is used
- Memo field (notes stored per material)

 Usage:

This section defines surface appearance and extra visual effects.

5. Custom Effects Settings section

Advanced shader controls:

- Specular intensity
- SPH reflection opacity

- Shadow brightness
- HDR intensity range
- Toon tone vector (color grading style)
- Multiple lights toggle
- Fog toggle

👉 Usage:

Used for fine-tuning stylized rendering behavior.

⚙️ Rendering Rules System

🧠 **ApplyCommonRules (important logic engine)**

This function runs whenever values change in the inspector.

It ensures that **shader behavior stays consistent with UI values**.

🔑 **What it controls:**

1. Alpha clipping

If alpha clipping is enabled:

- Enables shader keyword:
 - "ALPHATEST_ON"

👉 Meaning:

Pixels below a threshold become fully invisible (cutout effect).

2. Surface type

If surface is transparent:

- Enables keyword:
 - "SURFACE_TYPE_TRANSPARENT"

👉 Meaning:

Material is treated as transparent by rendering pipeline.

3. Blending mode behavior

Based on blend mode:

- Premultiply → disables alpha modulation
- Additive → disables alpha modulation
- Multiply → enables alpha modulation only in transparent mode
- Alpha → disables alpha modulation

👉 Meaning:

Controls how colors combine with background pixels.

4. Shadow receiving system

Controls a group of shadow-related flags:

If shadows are enabled:

- float properties are synchronized:
 - main light shadows
 - additional lights
- keywords are enabled:
 - shadow cascade
 - additional shadows

If disabled:

- everything is turned off

👉 Meaning:

Ensures shadows behave consistently in shader + Unity pipeline.

5. Dirty marking

Every material is marked as "dirty":

👉 Meaning:

Unity knows it must save/update the material.

🧰 Utility Function

🔧 **SetKeyword**

A small helper function:

- If enabled → turns shader keyword ON
- If disabled → turns shader keyword OFF

👉 Usage:

Used everywhere to simplify keyword management.

🔗 Shader-Specific Rules

ApplyShaderSpecificRules

Currently empty.

👉 Meaning:

Reserved for future shader-only logic.

🕒 How the whole system works together

1. Unity opens material inspector

↓

2. DrawUI builds all sections

↓

3. User changes values

↓

4. Base system detects changes

↓

5. ApplyCommonRules runs

↓

6. Shader keywords + floats are synchronized

↓

7. Material updates in real time

🧠 Key idea of this architecture

This inspector is not just a UI.

It is a **rule engine + UI hybrid**:

- UI lets users edit values
- Rules automatically enforce consistency
- Keywords keep shader behavior correct
- Base class ensures multi-material safety